Predictive Coding: What Is Exact, What Is Not, and What It Costs

Oruži* Zarathustra Goertzel

July 2026 — working note

Abstract

We prove what predictive coding (PC) does and does not compute, and test the consequences. Settling any finite linear-Gaussian residual model is exactly conditional Bayesian inference — the unique energy equilibrium *is* the posterior mean, at arbitrary width, depth, and correlated precision. The equilibrium error recursion is exactly backpropagation's, but the equilibrium weight gradient is not, and we give the counterexample. Exact backprop-equivalent scheduling is characterized completely: a schedule is exact iff the contributions it misses sum to zero, so when contributions cannot cancel it reduces to a structural criterion — every node's outgoing edges land on a single rank — strictly weaker than the levelizability one expects. Error-based PC is an exact nonlinear change of coordinates from state-based PC, with equal energy and local parameter updates; its first-arrival signal carries exactly the claimed λ^{L-i} factor. Composing that factor with the local update and loss laws proves a depth-credit no-go: inference rates equalize every layer only at the unit-rate backprop boundary, while polynomial Depth-μP scaling cannot cancel exponential attenuation. Settling on a depth-n path contracts at exactly $\cos(\pi/n)$. For a genuinely joined backbone–adapter path, restricting the coupled operator to a frozen backbone yields exactly the adapter-only first-mode rate; leaving the backbone active re-imports its depth bound. A preconditioned PC step multiplies the loss by $(P-1)^2$: descent holds exactly on $0 < P < 2$, only $P = 1$ reaches zero in one step, and asymptotically all that is required is $\rho(I-P) < 1$ — which a nonnormal Jordan family shows can coexist with arbitrarily large finite transients. The unrestricted conjecture that every saddle of equilibrated PC energy is strict is false: we give a critical cubic-order saddle with zero curvature in every direction. A separately trained centered-sigmoid PC model has exact and robust entropy turnover, while also proving contraction alone insufficient for reachability. Quadratic PC energy cannot orient even a two-node edge from any observational dataset; ordinary clamping is conditioning in both directions. A uniform intervention — clamp a node and delete its own residual term — separates the directions exactly when their covariance is nonzero. On any excited finite two-node intervention design, after both orientations minimize over their own structural coefficient, the true orientation has strictly lower energy if and only if that covariance is nonzero. Finally, two priors with *equal* cross-entropy can have strictly ordered expected search yields: fit does not imply guidance. All statements are machine-checked in Lean 4. The experiments separate regimes rather than forcing one verdict. End-to-end tree PC remained slower and below backpropagation in cumulative coverage, and the seq2seq ignition ladder required a backprop-equivalent PC endpoint before it departed. In contrast, a sealed two-world typed-GSLT continuation factorial, training only zero-output residual adapters from matched update-zero states, found 1,617 new OEIS entries in union: prospective PC had the largest update-rule union (968) and exclusive marginal (299), while remaining substantially slower and interacting strongly with carrier and seed. The result

*Oruži names the AI research collaboration responsible for formalization, implementation, experiments, and drafting.

is complementary value, not universal PC dominance. On a separately preregistered dependent-proof ranker, prospective PC starts at update zero, reproduces a strongly non-BP credit direction on two seeds (median raw cosines 0.0979 and 0.0930), and adds verified solved-set marginals despite trailing BP in total synthetic held-out coverage; unchanged incremental PC fails to learn. Mechanism separation and task efficacy are therefore measured independently.

Open artifacts. The formal repository, paper source, and compiled PDF are public. Every Lean link in this note is pinned to immutable commit `f3a31db36bb2f944312ee31c1d86c2758109e83a`; it opens the declaration in the source file rather than a search result or a moving branch.

1 Introduction

Predictive coding networks train by relaxing latent activities against local prediction errors, then applying purely local weight updates. The hope is a biologically plausible — and practically competitive — alternative to backpropagation, and the literature supplies equivalence results in the settling limit together with a growing engineering effort to make deep PC trainable at all [1, 2, 3, 4, 5].

Here is the picture to keep. Imagine a line of workers, each holding a guess about what the next worker should be seeing, with a meter between every pair showing the mismatch. Nobody sees the far end of the line. Learning happens in two phases: first the guesses jostle until the meters quiet down (settling), then each worker adjusts its habits using only its own adjacent meters (the local update). Backpropagation, by contrast, is a single messenger walking the whole line, telling each worker exactly how the final outcome would change if that worker alone shifted. The classical claim is that in the limit of patient settling, the jostling line and the messenger deliver the same instruction. Everything in this paper is about the fine print of that claim: which meter reading matches the messenger, under what timetable, how the required patience grows with the length of the line, and what all that jostling costs.

Those equivalence results are stated informally, and they are sloppiest exactly where the hypotheses matter: *which* quantity matches backpropagation, *under what schedule*, *at what depth*, and *at what cost*. This note answers those four questions with theorems, in models small enough that every hypothesis is explicit and every boundary carries a counterexample: linear chains, finite DAGs, scalar paths, finite categorical priors. Each is machine-checked in Lean 4 and axiom-clean, using only `propext`, `Classical.choice` and `Quot.sound`. Selected crown results display their source-exact Lean theorem headers in place, with proof bodies omitted; each caption links to the complete declaration and proof in an immutable source snapshot. Table 1 links every supporting statement and boundary fixture in the same snapshot.

Two of the answers surprised us. The natural conjecture about which graphs admit exact scheduling is false in an instructive direction (Section 5), and the question every PC-versus-BP comparison reports on — training loss — provably fails to determine the thing those comparisons care about (Section 11).

Section 12 reports the experiments the theory explains. Every arm fixed its acceptance criteria before running; failed arms are reported as failures with their measured failure modes. The companion report *AlienCoding* carries the full empirical account.

2 The model

A predictive-coding network attaches to each node i of a computation graph a latent state z_i, a prediction $f_i(z_{\mathrm{pa}(i)}; w)$ from its parents, and a precision $\lambda_i > 0$. The energy is

$$E(z, w) \;=\; \sum_i \lambda_i \left(z_i - f_i(z_{\mathrm{pa}(i)}; w)\right)^2.$$

Settling clamps the inputs (and, in supervised mode, the outputs) and descends E in z; the *weight update* is the local gradient $-\partial E/\partial w$ evaluated at the settled state. Backpropagation instead differentiates the supervised loss through the forward activations a. Every equivalence claim in the field is, underneath, a claim about the relationship between the settled state $z^\star$ and the forward state a.

Our running instance is the scalar chain of depth d: latents $z_0, \ldots, z_d$, links $i < d$ with gain w_i and precision λ_i, residuals and energy

$$r_i \;=\; z_{i+1} - w_i z_i, \qquad E(z, w) \;=\; \sum_{i<d} \lambda_i r_i^2,$$

clamped at $z_0 = x$ and $z_d = y$. We carry no factor of $\frac{1}{2}$, so the weight gradient is $\partial E/\partial w_i = -2\, e_i\, z_i$ where

$$e_i \;:=\; \lambda_i r_i$$

is the precision-weighted error at link i. The quantity e_i is the one that will turn out to be exact.

3 Settling is Bayesian inference

Read the chain as a linear-Gaussian generative model: each z_{i+1} is $w_i z_i$ plus noise of precision λ_i. Clamping z_0 and z_d conditions on the two ends.

The right level for this is not the chain. Take any finite *affine residual model*: latents u, residual $r(u) = Au + c$ with A injective, and an arbitrary positive-definite residual precision Λ — so residuals may be correlated, and A need not be square. The energy is $r(u)^\top \Lambda\, r(u)$.

Theorem 3.1 (Settling computes the posterior mean)**.** *For any such model the posterior precision $Q = A^\top \Lambda A$ is positive definite, and the following coincide: u is an energy equilibrium; u is the unique energy minimizer; and u is the conditional multivariate-Gaussian posterior mean. Any clamped state other than the posterior mean is provably not an equilibrium.*

Lean 4 signature and complete proof

```
theorem LinearGaussianOperatorModel.equilibrium_iff_eq_conditionalPosteriorMean
    (model : LinearGaussianOperatorModel Latent Residual)
    (u : LinearGaussianOperatorSpace Latent) :
    model.Equilibrium u ↔ u = ∫ v, v ∂model.posterior
```

So "settling performs Bayesian inference" is a theorem, not a slogan: the relaxed latents *are* the posterior mean given the evidence. Scalar chains and vector-valued chains — arbitrary width, arbitrary depth, arbitrary positive-definite (hence correlated) residual precision — are corollaries, obtained by exhibiting the chain's residual map as such an A and checking injectivity; no chain-specific completion-of-squares argument is repeated. At depth two the scalar case recovers the Kalman update and the two-source Gaussian fusion rule, which is where the PC literature's Bayesian intuition comes from. This fully-settled reading is the *prospective configuration* regime of [11], in which inference completes before the weights move; the Gaussian assumption it and we share can itself be relaxed without abandoning the inference view [17].

This is also the cleanest way to see the trouble in the next section. The posterior mean given a clamped output is precisely *not* the forward pass — conditioning on the answer moves the latents. That is the point of PC, and it is also why its weight gradient is not backpropagation's.

4 What matches backpropagation, and what does not

4.1 The error recursion is exact

Theorem 4.1 (Equilibrium errors obey backprop's recursion)**.** *At any clamped equilibrium of the scalar chain, for every i with $i + 1 < d$,*

$$e_i \;=\; w_{i+1}\, e_{i+1}.$$

This is backpropagation's backward recursion, exactly, with no limit and no approximation: errors travel from the output end to the input end multiplied by the intervening gains. Weight tying preserves it, which is the credit-assignment certificate for recurrent PC.

The same statement survives the move to the graphs real implementations use. Hash-consed evaluation DAGs share subexpressions, and the design question is whether a shared node gets one latent or one per use.

Theorem 4.2 (Shared-node aggregation)**.** *Let G be a finite DAG with one latent per node, edge gains w_e, node precisions λ_v, and errors $e_v = \lambda_v r_v$. At any equilibrium, every unclamped node v satisfies*

$$e_v \;=\; \sum_{e\,:\,\mathrm{src}(e)=v} w_e\, e_{\mathrm{tgt}(e)}.$$

That is reverse-mode accumulation: a shared node's settled error is the gain-weighted sum of the errors of everything it feeds, which is what differentiation does when it sums gradients over the unfolded tree. The negative fixture is the one that matters to implementors: on a graph where node 0 feeds node 1 through two edges, the correct settled error at node 0 is 2, whereas an implementation retaining a single parent slot records 1 — half the true value. One latent per shared node, all slots summed, is forced. Our C implementation of PC for the tree network (Section 12.1) is built this way, and its numerical oracles test these shapes.

4.2 The gradient identity is false

The frequently repeated stronger claim is that the equilibrium *weight gradient* equals the backprop gradient. It does not, and linearity does not save it.

Theorem 4.3 (Counterexample)**.** *Take the depth-2 chain with $w_0 = w_1 = 1$, $\lambda_0 = \lambda_1 = 1$, input $x = 1$ and clamped target $y = 2$. The forward state is $a = (1, 1, 1)$ and the clamped equilibrium is $z^\star = (1, \frac{3}{2}, 2)$. Then*

$$\frac{\partial E}{\partial w_0}\Big|_{z^\star} = -1 \qquad \textit{while} \qquad \textit{backprop's gradient for } w_0 = -2.$$

The root cause is $z_1^\star = \frac{3}{2} \neq 1 = a_1$.

Lean 4 signature and complete proof

```
theorem equilibriumGradient_ne_backpropGradient_example :
    pcGainPartial unitDepth2Links depth2EquilibriumState 0 ≠
      scalarSquaredErrorBackpropPartial 1 1 1
```

Both gradients use the correct error; they evaluate it against different source activations. Backprop multiplies by the forward activation a_1; PC multiplies by the relaxed latent $z_1^\star$, which conditioning has already moved toward the answer. On this fixture the two differ by a factor of two, and nothing about the linear case prevents it — by Theorem 3.1 the displacement $z^\star \neq a$ is exactly the Bayesian content of settling.

The consequence for the field is a naming discipline. What PC gets for free is the error recursion. The gradient identity has to be bought, and the price is a schedule.

5 Exact gradients live on a schedule — and only on some graphs

5.1 The schedule

Song et al.'s Z-IL result [2] recovers the exact gradient by timing rather than by settling. Index layers by distance from the output, so that forward consistency reads $w_i a_{i+1} = a_i$, let ρ be the output residual, and let backprop's errors be $\delta_0 = \rho$, $\delta_{i+1} = w_i \delta_i$, so that backprop's gradient for w_i is $\delta_i a_{i+1}$.

Theorem 5.1 (Exactness on schedule)**.** *If layer i's weights are read at inference step $t = i$ — the step at which the error front first reaches layer i, while its source latent still equals the forward activation a_{i+1} — then the scheduled PC update for w_i equals $\delta_i a_{i+1}$ exactly, for every $i < d$.*

The hypothesis is doing all the work. Read the same parameter at free equilibrium instead and you are back at Theorem 4.3. The identity is a property of *when* you look, not of the fixed point.

5.2 Which graphs admit one

Since exactness depends on reading each parameter exactly when its error arrives, the question becomes: on which graphs does "the first arrival" carry the *whole* error? Give each node a rank, let R be the head rank, and let the contribution travelling along edge e arrive at time $R - \mathrm{rank}(\mathrm{tgt}(e))$. Node v's first arrival is $\tau(v) = \min_{e\,:\,\mathrm{src}(e)=v}(R - \mathrm{rank}(\mathrm{tgt}(e)))$. Call the schedule *exact at v* when the contributions that have arrived by $\tau(v)$ already sum to the full aggregate.

It pays to separate two things the literature runs together: whether a schedule *numerically* gets the right number, and whether it *structurally* waited for everything. Call a node *admissible* at time t when every contribution it needs has arrived by t, and let the *missed aggregate* be the signed sum of the contributions that have not.

Theorem 5.2 (Exactness, classified)**.** *For any ranked DAG, any arrival map and any per-node read time, the schedule is exact at every node if and only if every node's missed aggregate is zero — equivalently, iff every node is either admissible or has its missed contributions exactly cancel. A schedule can therefore be exact while being inadmissible, and that happens precisely when some node's missed contributions cancel.*

Lean 4 signature and complete proof

```
theorem dagScheduleExactAtEveryNode_iff_each_admissible_or_cancellation
    (G : SharedLatentDAG Node Edge) (parentError : Node → ℝ)
    (arrival : Edge → ℕ) (nodeTime : Node → ℕ) :
    dagScheduleExactAtEveryNode G parentError arrival nodeTime ↔
      ∀ v,
        dagScheduleAdmissible G arrival (nodeTime v) v ∨
          dagMissedContributionsCancel G parentError arrival (nodeTime v) v
```

Cancellation is not hypothetical: a node missing contributions $+1$ and -1 is inadmissible and numerically exact at once. So "exact" and "waited for everything" are genuinely different properties, and only the second is structural.

They fuse exactly when cancellation is impossible — which is the regime the PC literature actually works in, since parent contributions there are strictly positive. Under that hypothesis exactness reduces to admissibility, and admissibility is decidable by rank alone. The natural conjecture at that point is unit levelizability: every edge steps one rank, $\mathrm{rank}(\mathrm{tgt}(e)) = \mathrm{rank}(\mathrm{src}(e)) + 1$, so everything arrives in lockstep. That is sufficient. It is not necessary, and the true criterion is weaker and more useful.

Theorem 5.3 (Exact schedules, characterized)**.** *Let G be a finite ranked DAG in which every parent contribution is strictly positive. Then the first-arrival schedule is exact at every node if and only if*

$$\forall v,\ \forall e_1, e_2 \text{ with } \mathrm{src}(e_1) = \mathrm{src}(e_2) = v: \quad \mathrm{rank}(\mathrm{tgt}(e_1)) = \mathrm{rank}(\mathrm{tgt}(e_2)).$$

That is: exactly when no node's outgoing edges mix target ranks.

Lean 4 signature and complete proof

```
theorem dagFirstArrivalScheduleExact_iff_uniformOutgoingTargetRanks
    (G : SharedLatentDAG Node Edge) (parentError : Node → ℝ)
    (headRank : ℕ) (hhead : ∀ n, G.rank n ≤ headRank)
    (hpositive : ∀ e, 0 < dagParentContribution G parentError e) :
    dagFirstArrivalScheduleExact G parentError headRank ↔
      HasUniformOutgoingTargetRanks G
```

Unit levelizability implies the criterion trivially — all targets sit at $\mathrm{rank}(v) + 1$ — but the converse fails, and we exhibit the witness: a graph with uniform long skips is *not* unit-levelized and *is* exactly schedulable. Skips are not the problem; *ragged* skips are. A residual connection in which a parameter feeds both a one-edge shortcut and a three-edge main path violates the criterion and is proven inexact, which is the mechanism behind Salvatori et al.'s observation that the construction is brittle on non-chain topologies [4].

There is also a value-independent signed classification. Group every missed edge by its target u, and let $g^{\mathrm{miss}}_{v,u}$ be the sum of its gains.

Theorem 5.4 (Structural signed exactness)**.** *A schedule is exact for* every *parent-error assignment if and only if $g^{\mathrm{miss}}_{v,u} = 0$ for every source v and target u. The same theorem holds for messages in any faithful real module. Thus signed skips may cancel structurally, but only within a target group; with target-injective nonzero gains, or common-sign gains, universal exactness reduces to admissibility.*

The cancellation witness is therefore not a lucky choice of error values: two missed edges to the same target, with gains $+1$ and -1, are universally exact. A single missed nonzero edge is universally inexact. The scalar and vector-valued fixtures prove both sides.

Read constructively, Theorem 5.3 is an architecture design rule:

> Skip connections are compatible with exact backprop-equivalent PC scheduling precisely when they do not mix target ranks. Uniform long skips are fine; rank-mixing residuals are not.

This is the one place where our theory tells an architect what to build rather than what to avoid, and it says the space of exactly-schedulable graphs is strictly larger than the field assumed.

6 Error coordinates: what ePC preserves

Goemaere et al.'s ePC [5] replaces latent states by prediction errors. For arbitrary nonlinear scalar layers f_i, fixed input x, and errors e_i, define recursively

$$z_0 = x, \qquad z_{i+1} = f_i(z_i) + e_i.$$

Conversely, any input-clamped state stream gives $e_i = z_{i+1} - f_i(z_i)$.

Theorem 6.1 (Exact state/error reparameterization)**.** *These constructions are mutually inverse. On every finite prefix the ePC energy equals the sPC energy after state reconstruction, and the corresponding local parameter update is identical. At the differential level,*

$$d(E_{\mathrm{sPC}} \circ z)_e = (dE_{\mathrm{sPC}})_{z(e)} \circ dz_e.$$

If dz_e has a continuous-linear right inverse, ePC and sPC have exactly the same critical-point condition.

This is a coordinate equivalence, not a blanket claim that every finite-step algorithm has the same trajectory. The initialization rate still controls how far a signal has travelled and how large it is when it arrives.

Theorem 6.2 (First-arrival attenuation)**.** *For the one-edge-per-step recurrence $s_0 = g$ and $s_{k+1} = \lambda s_k$,*

$$s_{L-i} = \lambda^{L-i} g.$$

If $|\lambda| < 1$, then for every positive tolerance the exact real signal eventually falls below it. It need not equal zero: a machine-checked fixture is nonzero while below tolerance. At $\lambda = 1$, a unit signal never falls below any tolerance at most one. A one-step ePC local update is λ times the backprop product, and is exactly that product at unit rate.

The tolerance statement is deliberately not called floating-point underflow. It is an exact real-number decay theorem plus an explicit external detection policy.

7 The cost of depth

Settling is relaxation, and relaxation on a long clamped chain is slow. This is folklore; here is the exact rate. On a path of n segments with both ends clamped, Jacobi settling is $(Jv)_k = \frac{1}{2}(v_{k-1} + v_{k+1})$ at interior nodes and 0 at the ends.

Theorem 7.1 (Exact relaxation rate)**.** *The mode $s_k = \sin(k\pi/n)$ satisfies $Js = \cos(\pi/n)\, s$, hence*

$$(J^t s)_k \;=\; \cos(\pi/n)^t \, \sin(k\pi/n) \qquad \textit{for all } t.$$

The spectral gap obeys $0 \le 1 - \cos(\pi/n) \le \pi^2/(2n^2)$, and $\cos(\pi/n) \to 1$ as $n \to \infty$.

Lean 4 signature and complete proof

```
theorem pathSlowMode_iterate_exact
    (segments : ℕ) (hsegments : 0 < segments) :
    ∀ t n, n ≤ segments →
      pathJacobiIterate segments (pathSlowMode segments) t n =
        pathRelaxationRate segments ^ t * pathSlowMode segments n
```

So the slowest mode decays by exactly $\cos(\pi/n)$ per sweep and the gap closes quadratically in depth: doubling depth quarters the progress per settling step. This is the mathematical spine under the engineering literature devoted to escaping it [5].

Real networks are not uniform, so the natural worry is whether one deep block poisons an otherwise shallow architecture. It does.

Theorem 7.2 (Heterogeneous blocks)**.** *For a finite family of blocks, each with its own depth n_i, coupling eigenvalue c_i and resulting exact mode rate, the family's modal spectral radius ρ satisfies $\rho < 1$ if and only if every block's rate is contractive; and if any block i has unit coupling, then*

$$1 - \rho \;\le\; \frac{\pi^2}{2\, n_i^2}.$$

The bound is the pessimistic one: the family's spectral gap is capped by the *deepest* unit-coupled block, no matter how shallow everything else is. Stability is a conjunction over blocks; speed is a minimum over blocks.

The corresponding escape is real, but only at an exact operator boundary.

Theorem 7.3 (Frozen-backbone restriction boundary)**.** *Join a backbone path of depth $B > 0$ to an adapter path of depth $A > 1$. The resulting length-$(B + A)$ block Jacobi operator is provably not block diagonal at the attachment. Now freeze by embedding an adapter state into that full operator, applying the full step, and projecting while deleting the clamped attachment row. The restricted operator is exactly the depth-A adapter Jacobi operator, and a scalar r is its first-sine-mode multiplier if and only if*

$$r = \cos(\pi/A).$$

Consequently its first-mode gap is independent of B and obeys $1 - r \le \pi^2/(2A^2)$. If the backbone instead remains active as a unit-coupled block, then the global modal gap again obeys the backbone bound $1 - \rho \le \pi^2/(2B^2)$.

Lean 4 signature and complete proof

```
theorem isFrozenAdapterSlowModeRate_iff
    (backboneSegments adapterSegments : ℕ)
    (hadapter : 1 < adapterSegments) (rate : ℝ) :
    IsFrozenAdapterSlowModeRate backboneSegments adapterSegments rate ↔
      rate = pathRelaxationRate adapterSegments
```

The independence is derived from restricting the coupled operator, rather than stored in an adapter definition. It is also sharp about the idealization: a depth-three backbone with a depth-two adapter has frozen adapter rate 0, but modal radius 1/2 when both unit-coupled paths remain active. Thus shallow adapters escape only when freezing really deletes the backbone coordinates from settling; partial freezing or a shared active relaxation can re-import the backbone depth.

The same first-arrival theorem also prices the imbalance that Depth-μP was designed to mitigate [7]. Let layer i be $L - i$ links from the output and let every inference link transmit at rate λ. Composing the sealed signal, one-step-update, and loss theorems gives the following result.

Theorem 7.4 (Depth-credit boundary)**.** *The effective per-layer preconditioner is*

$$P_i = \lambda^{L-i}, \qquad \ell_i^+ = (1 - \lambda^{L-i})^2 \ell_i.$$

For heterogeneous rates it is the suffix product $P_i = \prod_{j>i} \lambda_j$. Equalizing all P_i forces every used rate to equal one, because the output's empty product is one. Moreover, for $0 < \lambda < 1$, multiplying by any fixed polynomial in depth still tends to zero; exact cancellation requires the exponential reciprocal $\lambda^{-(L-i)}$.

Lean 4 signature and complete proof

```
theorem muPCDepthScaling_cannot_cancel_contractingDepthCredit
    (width depth : ℕ) (inferenceRate : ℝ)
    (hwidth : 0 < width) (hdepth : 0 < depth)
    (hrateNonzero : inferenceRate ≠ 0) (hrateContractive : |inferenceRate| < 1) :
    depthMuPAdamLearningRateScale width depth *
        depthMuPHiddenMultiplier width depth = 1 ∧
      (∀ distance,
        depthMuPAdamLearningRateScale width depth *
            depthMuPHiddenMultiplier width depth *
              inferenceRate ^ distance = inferenceRate ^ distance) ∧
      (∀ learningRate distance,
        learningRate * inferenceRate ^ distance = 1 ↔
          learningRate = (inferenceRate ^ distance)⁻¹) ∧
      (∀ coefficient degree,
        ∀ᶠ distance : ℕ in atTop,
          polynomialDepthCompensation coefficient degree distance *
              inferenceRate ^ distance ≠ 1)
```

The Depth-μP multiplier and its prescribed Adam learning-rate factor cancel each other algebraically, but neither changes the inference rate. Thus they repair a width parameterization, not this exponential credit factor; indeed the reported μPC experiments omit that Adam rescaling because it was empirically untrainable [7]. This is a scoped result, not a claim that μPC has no value: it identifies exactly which instability its parameterization cannot remove. The same module also resolves a nearby verbal dispute about whether equilibrated activities "barely move": in a uniform-precision unit chain, node k moves by exactly $(k/L)(y-x)$. Early fixed nodes move little as depth grows, while nodes near the output need not.

8 What a preconditioner can buy

The standard response to slow or misbehaving settling is to rescale the update — precision scaling, natural-gradient-style metrics [14], or a line search. In the exactly solvable case, what rescaling buys is a single number, and the answer is unexpectedly sharp.

Take one chain link: source activation s, downstream gain g, weight w, prediction gws, target y, residual $r(w) = gws - y$ and loss $\ell(w) = \frac{1}{2}r(w)^2$. Write $\gamma = gs$ for the effective gain and assume $\gamma \neq 0$. A PC update preconditioned by P moves along the normalized backprop direction, $w^+ = w - P\,\ell'(w)/\gamma^2$.

Theorem 8.1 (The $(P-1)^2$ law)**.** *$r(w^+) = (1-P)\,r(w)$, and therefore*

$$\ell(w^+) \;=\; (P-1)^2\,\ell(w).$$

Consequently, for $r(w) \neq 0$:

1. *$\ell(w^+) = 0$ if and only if $P = 1$;*
2. *$\ell(w^+) < \ell(w)$ if and only if $0 < P < 2$;*
3. *the shortfall against the ideal step is exactly $\Delta\ell(1) - \Delta\ell(P) = (P-1)^2\,\ell(w)$;*
4. *descent degrades strictly monotonically in the squared departure $(P-1)^2$.*

Every arm of our ignition ladder is a point on this parabola. An update in the correct direction at 0.5% magnitude is $P \approx 0.005$: it descends — it is inside $(0, 2)$ — while retaining $(0.995)^2 \approx 99\%$ of the loss per step. That is not a tuning failure, it is the law. And overcorrecting is worse than useless: $P = 3$ lies outside the descent window and *quadruples* the loss, which is what our precision-scaled arm did before it diverged. The window is $0 < P < 2$ and the target is $P = 1$, with quadratic penalty for missing.

The vector case is where the one-step picture must not be over-read. If the residual is an eigenvector of the preconditioner with eigenvalue μ, the one-step law applies with μ in place of P: $\ell \mapsto (\mu - 1)^2 \ell$. It is tempting to conclude that a preconditioner is correct only where its spectrum sits at 1. That reading is wrong, and the asymptotic truth is more permissive.

Theorem 8.2 (Convergence is a spectral-radius condition)**.** *Write $D = I - P$ for the preconditioner's departure from the identity. In any complete normed algebra, $\rho(D) < 1$ if and only if $D^n \to 0$; hence the preconditioned residual iterate converges to zero exactly when $\rho(I - P) < 1$. No diagonalizability is assumed, so nonnormal operators are covered.*

Lean 4 signature and complete proof

```
theorem spectralRadius_lt_one_iff_pow_tendsto_zero
    {A : Type*} [NormedRing A] [NormedAlgebra ℂ A] [CompleteSpace A]
    [NormOneClass A] [Nontrivial A] (a : A) :
    spectralRadius ℂ a < 1 ↔
      Tendsto (fun n : ℕ => a ^ n) atTop (𝓝 0)
```

So $P = I$ is the *one-step optimum*, not the requirement. Asymptotically an entire open region works, and one may sit far from the identity on every mode and still converge. For a scalar this is the familiar $|1 - P| < 1$ — the window $0 < P < 2$ of Theorem 8.1 — and the spectral form is what survives to matrices.

Finite-time behavior needs its own theorem.

Theorem 8.3 (Geometric envelope)**.** *If $\rho(D) < q < 1$, then there is a finite $C \geq 1$ such that*

$$\|D^n\| \leq Cq^n \quad \text{and} \quad \|D^n r\| \leq Cq^n \|r\| \qquad (n \geq 0).$$

The constant absorbs the finite nonnormal transient; it cannot in general be determined from the spectral radius alone.

The obstruction is exact for the Jordan family

$$D_{\lambda,c} = \begin{pmatrix} \lambda & c \\ 0 & \lambda \end{pmatrix}, \qquad D_{\lambda,c}^n = \begin{pmatrix} \lambda^n & nc\lambda^{n-1} \\ 0 & \lambda^n \end{pmatrix}.$$

Theorem 8.4 (Arbitrarily large stable transients)**.** *$D_{\lambda,c}$ has spectrum $\{\lambda\}$, and is nonnormal whenever $c \neq 0$. For $\lambda = 0$ it satisfies $D^2 = 0$, yet a unit residual is amplified on the first step by exactly $|c|$. Consequently, for every proposed bound $B \geq 0$ there is a nonnormal operator with spectral radius zero, exact two-step convergence, and first-step amplification greater than B.*

The earlier witness $c = 2$ is therefore one point in an unbounded family, not a curiosity. Asymptotic stability, geometric rate, transient size, and step-wise descent are four different claims. A nonnormal preconditioner may honour the first two while violating the fourth, with the third controlled only by additional operator information.

What no amount of spectral latitude buys is a single global knob.

Theorem 8.5 (One preconditioner cannot serve two modes)**.** *There is a two-mode fixture and a single diagonal preconditioner* $\mathrm{diag}(\frac{1}{2}, 3)$ *that strictly* decreases *the loss on one eigenmode and strictly* increases *it on the other, from equal starting losses.*

This is the no-go for scaling one's way out: a metric that is merely large, or merely uniform, buys nothing, because its eigenvalues are wrong in both directions at once. The bar is spectral rather than magnitudinal — but by Theorem 8.2 it is a bar of membership, $\rho(I - P) < 1$, not of identity. Information-geometric preconditioning therefore keeps an honest target that is not merely "be a second-order method".

Line search fares better, because it targets P adaptively rather than fixing it.

Theorem 8.6 (Backtracking descends)**.** *Under an explicit smoothness bound, Armijo backtracking terminates — some* $P = P_0\sigma^n$ *satisfies the Armijo condition — and every accepted step is non-increasing in energy. This instantiates with the exact smoothness constant of the depth-2 PC energy.*

This is the correctness certificate for the settling procedure used by the surviving experimental arms, whose logged energies were monotone in every accepted run.

Two later formal tranches sharpen what a preconditioner claim may assert. The first is Fisher geometry: for a Gaussian layer the weight Fisher is exactly the Kronecker product of the precision with the activation second moment (`gaussianWeightFisher_eq_kronecker`), so "precision-weighted predictive coding is a natural-gradient method" is a conditional statement, and the condition fails concretely—under correlated activations no scalar precision recovers the natural step (`scalarPrecision_cannot_recover_correlatedNaturalStep`). Between the full Fisher and a scalar precision sits a strict approximation ladder (layer block, Kronecker, activation-only, diagonal, scalar; `fisherApproximation_strict_ladder`) with an explicit direction-error bound (`direction_error_le_approximate`), and a composed error budget separates settling, equilibrium, covariance, damping, and outer-optimizer contributions—so an experimental arm can be adjudicated as PC-distinct yet not Fisher-natural, which is precisely the classification our scalar-precision-plus-Adam arms occupy.

The second welds this paper's two surviving mechanisms into one object: a depth-indexed joint energy with hidden error coordinates *and* an explicit prospective output state. Zeroing the hidden errors recovers exactly the output-only prospective objective; hard-constraining the output to the frozen prediction recovers exactly the deep error-coordinate objective (`prospectiveEPCJointEnergy_zeroErrors_eq_outputObjective`, `prospectiveEPCJointEnergy_hardOutputConstraint_eq_deepObjective`). The join has an ignition boundary—with a zero readout and zero hidden errors, every hidden force vanishes while the output force persists (`zeroReadout_zeroErrors_hiddenForce_eq_zero`), so a zero-initialized adapter provably stages itself: output first, depth after. And the two mechanisms may not be naively combined: summing their independently computed parameter updates is an effective overshoot that strictly increases the shared energy in an explicit fixture. The joint rule, not the sum, is the candidate worth testing.

9 Not every equilibrated-energy saddle is strict

Innocenti et al. prove strictness for important classes of deep-linear equilibrated PC energy and conjecture that every remaining saddle is strict [6]. A strict saddle is a critical point with a direction of negative second curvature. The unrestricted conjecture has a covariance-degenerate boundary that the partial theorem does not cover.

Take a scalar two-layer model and the two examples with inputs $(1, 1)$ and targets $(1, -1)$. Its

exact equilibrated energy at weights (a, b) is

$$F(a, b) = \frac{1 + a^2 b^2}{2(1 + b^2)}.$$

This formula is derived in Lean from the rescaled batch residual objective, rather than postulated. Every $(a, 0)$ is critical because the input–target covariance vanishes.

Theorem 9.1 (Strict region and non-strict boundary)**.** *At $(a, 0)$, the output-weight direction has second curvature $a^2 - 1$. Hence every $a^2 < 1$ has a strict-saddle certificate, including the published origin case. But at $(1, 0)$ every directional second curvature is zero, while for every $0 < t < 1$,*

$$F(1 - t, t) < F(1, 0) < F(1 + t, t).$$

Thus $(1, 0)$ is a genuine saddle with explicit descending and ascending approach curves, but it is not strict.

Lean 4 signature and complete proof

```
theorem orthogonalDataset_universalStrictSaddleConjecture_counterexample :
    IsCriticalPoint orthogonalDatasetEquilibratedEnergy (1, 0) ∧
      HasTwoSidedSaddleCurves orthogonalDatasetEquilibratedEnergy (1, 0)
        (fun step => (1 - step, step))
        (fun step => (1 + step, step)) ∧
      (∀ direction,
        HasSecondDirectionalCurvatureAt orthogonalDatasetEquilibratedEnergy
          (1, 0) direction 0) ∧
      ¬ HasStrictSaddleCertificate orthogonalDatasetEquilibratedEnergy (1, 0)
```

So the universal conjecture is false without a nondegeneracy hypothesis. This does not contradict the published strictification theorem: the counterexample lies outside its covered class. It identifies the missing issue more sharply — a cubic-order saddle can survive exactly where the quadratic curvature vanishes.

10 The coupled system

Settling and learning run together, and their interaction has its own stability region. Write the two-timescale linear system with settling precision λ and learning rate η: the settled error is multiplied by $1 - \lambda$ per step, and the weight by $1 - \eta\lambda$.

Theorem 10.1 (Stability region)**.** *Both multipliers are contractive if and only if*

$$0 < \lambda < 2 \qquad \textit{and} \qquad 0 < \eta\lambda < 2,$$

in which case the coupled iteration converges to zero from every initial state. If $\eta\lambda > 2$, the learning eigenmode diverges in norm.

Note the shape: $0 < \eta\lambda < 2$ is the same window as $0 < P < 2$ in Theorem 8.1, which is not a coincidence — both are the condition for a one-step affine contraction. It is also the formal account of the late-run divergence one of our arms exhibited: the product $\eta\lambda$, not either factor, is what must stay inside the window.

The second dynamical fact concerns what settling does to the *shape* of the output distribution, and it is the one our experiments measured directly.

Theorem 10.2 (Entropy turnover)**.** *For a linear signal of any depth $d > 0$ and any soft target $p \in (0, 1)$ with $p \neq \frac{1}{2}$, the settled output's entropy $H(s)$ after s settling steps satisfies*

$$H(0) > H_b(p), \qquad H(d) = H_b(p), \qquad H(d+1) > H_b(p),$$

where H_b is binary entropy. Hence H is neither monotone nor antitone; and it is non-monotone if and only if *the target is non-uniform. A gradient-trained comparator provably lands strictly below the target entropy.*

Under-settle and the output is diffuse; settle exactly and you hit the target; over-settle and it re-diffuses past it. Settling depth is therefore not a "more is better" knob: there is an interior optimum, and it is at the signal depth. Our tree-network campaign observed this curve — including the turnover — before we proved it.

The turnover is not an artifact of linearity. Let the settling map be any monotone contraction fixing zero, with rate $q < 1$ — a class that includes the saturating nonlinearities PC implementations actually use.

Theorem 10.3 (Nonlinear turnover, sufficient conditions)**.** *Fix such a settling map, a depth $d > 0$, and a non-uniform target $p \in (0, 1)$, $p \neq \frac{1}{2}$. If the depth-d iterate reaches the target logit from the seed, then the same three regimes hold — entropy above target before arrival, equal at depth, above again afterwards — and the entropy is neither monotone nor antitone. Moreover contraction bounds the reachable set: $|\operatorname{logit}(p)| \leq q^d\,|seed|$.*

The reachability hypothesis is doing real work and we flag it rather than bury it: this is a theorem about a class of nonlinear settling dynamics that *can* hit their target, not a theorem about arbitrary trained nonlinear PC networks. The contraction bound is the honest sting — it says which targets a given depth can reach at all. We therefore build and train one concrete nonlinear model rather than assume reachability.

Let $\phi(x) = \sigma(x) - \frac{1}{2}$. This centered sigmoid fixes zero, is strictly increasing and globally $\frac{1}{4}$-Lipschitz, and is proved not to agree pointwise with any real-linear map. Put a parameter w one edge upstream, read $p_w = \sigma(\phi(w))$ after one settling step, choose p_1 as the training target, and train with $L(w) = (p_w - p_1)^2$.

Theorem 10.4 (Trained nonlinear turnover)**.** *$w = 1$ is the unique zero-loss parameter and a global minimizer. At that trained weight, the output probabilities at settling depths $0, 1, 2$ are $\frac{1}{2}, p_1, \frac{1}{2}$, with $p_1 > \frac{1}{2}$, so entropy strictly decreases and then increases. The middle entropy remains strictly below uniform throughout an explicit open probability neighborhood of p_1. The dynamics lift coordinatewise to every finite vector width; an all-one seed gives turnover in every coordinate, while an all-zero seed stays uniform.*

Lean 4 signature and complete proof

```
theorem trainedNonlinearEntropyTurnover_crown :
    (¬ ∃ L : ℝ →ₗ[ℝ] ℝ, ∀ x, L x = centeredSigmoidActivation x) ∧
      trainedCenteredSigmoidProblem.loss 1 = 0 ∧
      (∀ weight, trainedCenteredSigmoidProblem.loss 1 ≤
        trainedCenteredSigmoidProblem.loss weight) ∧
      Real.binEntropy trainedCenteredSigmoidTargetProbability <
        trainedCenteredSigmoidProblem.entropy 1 0 ∧
      trainedCenteredSigmoidProblem.entropy 1 1 =
        Real.binEntropy trainedCenteredSigmoidTargetProbability ∧
      Real.binEntropy trainedCenteredSigmoidTargetProbability <
        trainedCenteredSigmoidProblem.entropy 1 2 ∧
      (¬ ∃ seed : ℝ, centeredSigmoidSettling.activation seed = 1) ∧
      0 < trainedCenteredSigmoidRobustRadius
```

This separates three notions that "contractive nonlinear PC" can obscure. Contraction controls sensitivity, training certifies the chosen target, and the one-edge delay causes turnover. None implies the others: the same centered sigmoid is contractive but its range is $(-\frac{1}{2}, \frac{1}{2})$, so no seed can reach logit 1.

11 Fit does not imply guidance

The last result is not about PC at all. It is about what PC-versus-BP comparisons measure, and it is the reason our empirical verdict is stated in the currency it is.

A search-guidance prior θ over k program tokens is sampled i.i.d. within a budget B; a *hit* is a draw of the target.

Theorem 11.1 (Yield is target mass)**.** *The expected number of target hits is exactly $B\,\theta_{\text{target}}$, and is strictly increasing in θ_{target}.*

That is unsurprising, and it is the setup for the separation.

Theorem 11.2 (Equal loss, ordered yield)**.** *Let $\theta^{\text{lo}} = (\frac{3}{4}, \frac{1}{4})$ and $\theta^{\text{hi}} = (\frac{1}{4}, \frac{3}{4})$, with outcome 1 the target. Against the soft target $(\frac{1}{2}, \frac{1}{2})$ these priors have* equal *cross-entropy, yet for every budget $B > 0$ their expected yields are strictly ordered:*

$$\frac{B}{4} \;<\; \frac{3B}{4}.$$

Lean 4 signature and complete proof

```
theorem equal_crossEntropy_strictly_ordered_expectedYield :
    softHalfTargetCrossEntropy lowTargetMassPrior =
        softHalfTargetCrossEntropy highTargetMassPrior ∧
      ∀ budget : ℕ, 0 < budget →
        iidExpectedTargetHits lowTargetMassPrior budget 1 <
          iidExpectedTargetHits highTargetMassPrior budget 1
```

Two models, identical loss, threefold difference in search performance. Cross-entropy is symmetric under swapping mass between outcomes; yield is not, because only one outcome is the target. So training loss — the quantity every PC-versus-BP comparison in the literature reports, including ours — provably does not determine search performance.

What does determine it, in this model, is the mass the prior puts on the target: its *decisiveness*. This is why Section 12.1 reports prior entropy alongside loss, and why the entropies are the numbers that moved with performance.

12 The experiments

12.1 Paired self-learning campaign on the tree network

We reproduced Gauthier's OEIS program-synthesis pipeline byte-faithfully (Standard ML, HOL4, tree neural network in C) and implanted PC *in place* in the C training code, guarded by numerical oracles (analytic golden vectors to 10^{-9}–10^{-16}; the BP path byte-identical when PC is off). The learning rule is the only variable.

Single-generation results establish the mechanism. PC's yield climbs with settling depth — $63\%, 67\%, 83\%, 93\%$ of BP's newly solved sequences at $4, 8, 16, 32$ settling steps — at roughly linear cost, up to $\sim 10\times$ BP's training time at 32 steps, and then *turns over* at 64 steps. Throughout, the entropy of the search-guidance prior tracked yield at every point, including the turnover, while training loss did not: PC fit the training objective as well as or better than BP while guiding search worse. Measured prior entropies: BP 0.671; PC $1.479 \to 0.975$ over steps $4 \to 32$, re-diffusing to 1.154 at step 64. Theorem 10.2 is the shape of that curve and Theorem 11.2 is why loss failed to see it.

The multi-generation campaign makes the causal comparison real: three paired seed-worlds, one frozen random generation-0 each, both arms grown for five self-training generations under identical budgets. Cumulative solved OEIS sequences:

	gen-1	gen-2	gen-3	gen-4	gen-5
BP, world 1	8,752	11,709	13,224	14,566	15,852
BP, world 2	8,361	11,291	13,250	15,054	16,081
BP, world 3	8,627	11,510	13,267	14,793	15,843
PC, world 1	6,465	6,914	9,067	9,486	10,211
PC, world 2	6,093	8,031	8,359	9,195	9,411
PC, world 3	7,548	7,548	8,820	10,111	10,296
PC/BP ratio	78%	65%	66%	65%	62.6%

BP compounds metronomically: the three endpoints land within 1.5% of one another (15,843–16,081). PC ignites but advances in erratic bursts — including one genuine zero-progress generation

(world 3, generation 2, byte-identical solution set) — and finishes between 58.5% and 65.0% of BP's frontier (world 1 64.4%, world 2 58.5%, world 3 65.0%), 62.6% in aggregate. Same frozen start, same search budget: the learning rule alone costs a third of the frontier, at 32 settling steps' training overhead.

Unique coverage: does PC find solutions BP does not? Yes, and the answer has a shape. Per world, PC solves 242/277/286 sequences its paired BP arm misses ($\approx$ 2.5% of PC's own catalogue, against $\sim$ 6,000 the other way). Pooling all three worlds, PC's union contributes 246 sequences that *no* BP world solved — a combined catalogue of 19,280 against BP's 19,034. One case is reproducible rather than stochastic: a single sequence (`A087299`) solved by PC in all three independently evolved worlds, by BP in none, with the *identical* 11-token program each time — an existence proof that the two rules steer search into genuinely different territory. The mechanism is visible in program lengths (first-listed program per solved sequence, pooled over worlds): the sequences only BP solved take median-24-token programs (mean 32.4), the sequences only PC solved median 19 (mean 20.7), and at the generation-five frontier the gap widens — BP's newly solved sequences run to median 40 tokens (mean 47.3) against PC's 25 (mean 28.1). Decisive guidance sustains long derivations; flatter guidance spreads mass across short ones and occasionally covers a short solution that confident-but-elsewhere mass walks past. The economics, however, are one-sided: adding a *second BP world* contributed 2,241 new sequences and a third 941, so as a diversity source PC is dominated several-fold by re-running BP on a fresh seed — on this topology. Whether richer topologies (residual adapters on a frozen backbone — the one regime the depth theorems endorse — or graph-structured models) change PC's standing is deliberately left open and would require a separately registered comparison.

Campaign closure and provenance. All thirty phase artifacts (three worlds $\times$ two arms $\times$ five guided generations) are sealed with per-file SHA-256 evidence manifests, and every number in this subsection was recomputed from those sealed artifacts before publication. The zero-progress generation is hash-verified: world 3's PC solution file carries the identical digest at generations one and two. Four operational amendments were filed during the run — memory-floor recovery and driver-relaunch policy — each sealed against the campaign seal and each recording *protocol unchanged, science parameters unchanged*; the arms, the checker, and the validation path were never touched. The campaign halted at generation five, the pre-declared mandatory guided generation, by an operator decision recorded before the final arms completed; the protocol's adaptive stopping rule (earliest adaptive stop at generation seven, hard stop at ten) was consequently never evaluated, and no claim here rests on it. Extending the same frozen worlds toward the hard stop remains available as a pre-registered fork.

12.2 The seq2seq ignition ladder

On a faithful reimplementation of the alien-coding NMT baseline [10] (14-token program language, 512-unit bidirectional-LSTM encoder–decoder with scaled-Luong attention; the trained reference solves 1,860 new OEIS sequences on seed 1, with 222 target-associated hits against a randomized-association null of 6), we ran a seven-arm overfit-32 ignition ladder for PC variants. The registered composite bar: final loss ≤ 0.05; at least 30/32 exact greedy decodes; monotone settling energies; and *distinctness* — the final updates must differ measurably from BP's, median parameter cosine < 0.95 — so that a pass cannot be secretly backpropagation. The BP anchor passes at loss 0.026, 32/32.

arm	result	loss / exact	measured failure mode
fixed-rate ePC	FAIL	105.4 / 0	settling divergence
backtracking, 16-step	FAIL (time)	—	exceeded registered wall-clock
backtracking, 5-step	FAIL	11.2 / 1	BP-direction updates at 0.5% magnitude
precision-scaled	FAIL	21.4 / 0	late-run divergence
staged BP→PC, 4000	FAIL	0.075 / 32	loss bar only; distinctness passed
staged BP→PC, 4500	FAIL	0.044 / 32	distinctness only; updates BP-like
staged, deepened inference	**PASS**	0.037 / 32	all bars; median cosine 0.80

Three of these failures are theorems. The 5-step arm is $P \approx 0.005$ in Theorem 8.1: correct direction, 99% of the loss retained per step. The precision-scaled arm left the $0 < \eta\lambda < 2$ window of Theorem 10.1. The fixed-rate arm is the same failure without the line search that Theorem 8.6 certifies.

The joint reading of the two staged arms is the sharpest empirical fact in this note: the staged schedule that starts at provable BP-equivalence and walks toward PC *learns to the bar exactly as far as its updates remain BP-like*, and retains PC-distinctness exactly when it misses the bar. The seventh arm broke the tension: continuing the staged run with deepened inference passed every registered bar at once — loss 0.0366, 32/32 exact decodes, monotone settling, genuinely PC-distinct updates (median parameter cosine 0.80, 13/18 tensors below 0.95).

That pass was seed 0 and judged at one terminal snapshot, so a protocol weakness had to be settled before it could be believed: the terminal cosine statistic turns out to swing *within a single fixed stage* — measured medians $0.929, 0.952, 0.322, 0.787, 0.801$ at updates 4501 through 4700 — so no one endpoint is a robust mechanism criterion. The replication was therefore pre-registered on an independent seed and on a prospectively fixed five-checkpoint window instead, with the bars unchanged.

It passes. On seed 1, the complete staged schedule (3000 BP-equivalent one-step anchor updates, then fixed transitions to five-step inference, 4700 updates in total) meets every registered bar: final loss 0.0014, 32/32 exact decodes, all 96 logged inference records monotone, and mechanism distinctness at *all five* checkpoints — median of the checkpoint medians 0.228, against a required cutoff of 0.95. The updates are thus far *more* PC-distinct than seed 0's terminal 0.80, and distinct throughout the window rather than at a chosen point. Staged ePC ignition reproduces.

Two scope limits are registered with it and remain: the schedule begins from a BP-equivalent anchor, so this is not learning from random initialization; and it is overfit-32 scale, not full-scale NMT performance.

12.3 Typed-GSLT continuation from matched update-zero adapters

A later experiment asks a different question from both campaigns above. Four fully frozen generation-eight typed-GSLT parents—a recurrent vector, typed workspace, and two belief-register carriers—receive the same zero-output residual adapter. Within each parent and world, five update rules start from identical model state, data ledger, minibatch order and search budget: ordinary BP, an independent BP replication, deep error-coordinate PC, incremental PC, and prospective PC. Generation ten continues each generation-nine adapter and optimizer exactly; it does not reinitialize or pass through a BP-pretraining phase. The registered surface is two worlds, four carriers, five rules and two continuation generations. Every proposed program is checked against the whole OEIS.

The checker-audited cross-world union contains 1,617 new A-numbers. Grouping the same sealed cells by update rule gives:

update rule	union	found by no other rule
prospective PC	968	299
error-coordinate PC	893	66
ordinary BP	869	51
independent BP	866	77
incremental PC	693	124

Prospective PC therefore has both the largest union and, by a wide margin, the largest exclusive marginal. It does not win every matched cell. In world two it exceeds ordinary BP by 38 entries on the recurrent carrier, 18 on derived belief and 12 on learned belief, and ties on the workspace; in world one it leads by 53 on learned belief, ties derived belief, and trails on the recurrent and workspace carriers. BP is substantially faster per reported training hour. The warranted conclusion is that prospective PC contributes real, architecture-dependent search diversity in this frozen-parent adapter regime. Neither the aggregate ordering nor the name of the rule licenses a universal efficacy claim.

The provenance boundary matters. Shared global checker channels allowed orphan workers to overwrite supporting checker outputs. The original numerical accountings were therefore replayed through seven fresh single-owner checker runs. The ordinary and audited numerical summaries agree exactly, while the authoritative terminal verifier retains the isolated evidence chain and passes all 749 independent checks. Its SHA-256 is

`3f08e6c3a57ba3e13a0e387737bbb31cfd990221a7b3b331214b4f9cab47bc7f.`

The audited verdict it binds has SHA-256

`02ca3cb75003f18678c5fcd43d93c07da584a365ffec709510a5c7147f8491a6.`

These hashes identify the result; they do not turn experiment data into a theorem.

A subsequent OEIS branch[1] crossed three carriers—recurrent, typed workspace and the new unified routed—with two BP controls and three PC rules (prospective, the new multi-site error-coordinate, and incremental), all rules starting from identical generation-eight parent bytes, PC active from the branch's first update, 2,500 updates per cell per round, three rounds, one seeded world, same per-cell search budget and whole-OEIS checker. Persistent cross-episode evidence is disabled in the unified carrier. Round three was fixed as the branch's terminal resource boundary before its result was available; the recorded counts below are admitted only after the round-three seal and an independent recount.

Mechanism telemetry keeps direction and magnitude separate. Raw-credit cosine records the local direction before optimizer transport, while optimizer-displacement cosine records the direction actually transported to the parameters; neither statistic alone controls the transported step size. Indeed, both cosines can equal one while the transported norm is only 10^{-3} of the matched BP norm (`cosine_eq_one_not_normRatio_le`). Accordingly, the terminal tables report raw and transported cosine together with their respective norm ratios. Verified discoveries per unit work, rather than cosine alone, remain the efficacy endpoint.

[1] A botched experiment that nonetheless adds data. The branch was intended as a from-scratch comparison of update rules; it was mis-specified and executed as a continuation from generation-eight weights (the last rule-neutral common ancestor, trained by the earlier backpropagation loop) over the generation-ten single-world pool. Its verified discoveries enter the solution pool with full provenance; its update-rule comparisons are single-world portfolio observations on a BP-built stack, recorded for completeness, and are not evidence on the from-scratch question. Row labels 11–13 are pool-accounting indices, not scientific generation numbers; the branch is parallel to, not a successor of, the generation-nine/ten factorial.

12.4 Recorded branch outcome (tabulated, not interpreted)

Union 747 (BP family 673, PC family 452, PC-only 74, Jaccard 378/747 ≈ 0.506, PC work 6.849× BP). Within the branch the independent-BP control is the largest single arm (400) and incremental PC fails (56):

rule	round 1	round 2	round 3	cumulative	exclusive	train h
bp_control	198	118	62	378	7	4.227
bp_independent	234	102	64	400	287	4.133
incremental_pc	27	12	17	56	43	39.159
multi_site_error_coordinate_pc	199	118	64	381	17	12.857
prospective_pc	202	115	66	383	7	5.242

Pairwise coverage: seed identity moves coverage more than rule identity (same-seed cross-rule Jaccard up to 0.9364; cross-seed same-family 0.1560):

rule A	rule B	$\|A\|$	$\|B\|$	$\|A \cap B\|$	A-only	B-only	Jaccard
bp_control	bp_independent	378	400	105	273	295	0.1560
bp_control	incremental_pc	378	56	7	371	49	0.0164
bp_control	multi_site_error_coordinate_pc	378	381	356	22	25	0.8834
bp_control	prospective_pc	378	383	368	10	15	0.9364
bp_independent	incremental_pc	400	56	8	392	48	0.0179
bp_independent	multi_site_error_coordinate_pc	400	381	100	300	281	0.1468
bp_independent	prospective_pc	400	383	105	295	278	0.1549
incremental_pc	multi_site_error_coordinate_pc	56	381	7	49	374	0.0163
incremental_pc	prospective_pc	56	383	7	49	376	0.0162
multi_site_error_coordinate_pc	prospective_pc	381	383	361	20	22	0.8958

Opportunity control: an independent BP seed given identical opportunity weakly dominates all nine candidate arms on the count/work Pareto (`insert_candidate_gt_insert_opportunity_iff` instantiated):

carrier	rule	ext.	indep. BP ext.	beyond 2 BP	work ×BP	BP dom.
gru	incremental_pc	10	130	10	7.445	yes
gru	multi_site_error_coordinate_pc	8	130	8	1.330	yes
gru	prospective_pc	1	130	0	1.073	yes
typed_workspace	incremental_pc	4	94	4	8.945	yes
typed_workspace	multi_site_error_coordinate_pc	11	94	11	2.845	yes
typed_workspace	prospective_pc	5	94	5	1.159	yes
unified_routed	incremental_pc	36	156	34	10.524	yes
unified_routed	multi_site_error_coordinate_pc	9	156	9	3.879	yes
unified_routed	prospective_pc	11	156	11	1.414	yes

The final-round follow-up of every PC arm that crossed the registered two-round boundary is:

carrier	rule	new	excl.	only vs BP	beyond two BP	coverage/BP	train h
gru	incremental_pc	1	0	1	1	0.032	1.603
gru	multi_site_error_coordinate_pc	31	3	3	3	1.000	0.280
gru	prospective_pc	32	0	1	0	1.032	0.226
typed_workspace	incremental_pc	2	2	2	2	0.091	4.721
typed_workspace	multi_site_error_coordinate_pc	22	1	2	2	1.000	1.459
typed_workspace	prospective_pc	23	0	1	1	1.045	0.686
unified_routed	incremental_pc	14	12	13	12	0.737	7.799
unified_routed	multi_site_error_coordinate_pc	16	1	1	1	0.842	3.084
unified_routed	prospective_pc	21	3	3	3	1.105	1.223

The corresponding direction-and-magnitude telemetry is:

carrier	rule	raw cos.	raw norm	opt. cos.	opt. norm	first opt.	last opt.	< .995	< .95
gru	incremental_pc	1.0000	0.999761	0.9891	0.003951	0.9892	0.9890	1	–
gru	multi_site_error_coordinate_pc	1.0000	0.999473	1.0000	0.999911	1.0000	1.0000	–	–
gru	prospective_pc	1.0000	0.999477	1.0000	0.999911	1.0000	1.0000	–	–
typed_workspace	incremental_pc	1.0000	0.999831	0.9886	0.001969	0.9886	0.9887	1	–
typed_workspace	multi_site_error_coordinate_pc	1.0000	0.999664	1.0000	0.999940	1.0000	1.0000	–	–
typed_workspace	prospective_pc	1.0000	0.999681	1.0000	0.999944	1.0000	1.0000	–	–
unified_routed	incremental_pc	1.0000	0.999832	0.9964	0.000980	0.9968	0.9961	5	–
unified_routed	multi_site_error_coordinate_pc	1.0000	0.999656	1.0000	0.999937	1.0000	1.0000	–	–
unified_routed	prospective_pc	1.0000	0.999668	1.0000	0.999941	1.0000	1.0000	–	–

The compute dynamics for those final-round PC cells are:

gen.	carrier	rule	train h	infer steps	interleaved	infer backtracks	local backtracks	peak MiB
13	gru	incremental_pc	1.603	5000	5000	0	45555	166.5
13	gru	multi_site_error_coordinate_pc	0.280	5000	2500	0	0	146.3
13	gru	prospective_pc	0.226	5000	2500	0	0	130.0
13	typed_workspace	incremental_pc	4.721	5000	5000	0	50883	737.6
13	typed_workspace	multi_site_error_coordinate_pc	1.459	5000	2500	0	0	504.3
13	typed_workspace	prospective_pc	0.686	5000	2500	0	0	305.6
13	unified_routed	incremental_pc	7.799	5000	5000	0	54988	1011.3
13	unified_routed	multi_site_error_coordinate_pc	3.084	5000	2500	0	0	676.3
13	unified_routed	prospective_pc	1.223	5000	2500	0	0	396.1

The fixed-budget retention channel completed 1800 records and submitted 54935 candidates, recovering 0 exact programs and 0 semantic-prefix reproducers; timeout affected 0 cells. This channel measures teacher-forced, fixed-budget behavioural retention only; a null recovery count bounds neither cross-task transfer nor what an unconstrained search would recover, and it is not evidence that the retained knowledge is absent from the weights. The analysis and independent verification are bound by SHA-256 identifiers `3d79d72cfe3cf2ce280e89a3cf1ab3fd5290bf4bd42d98ba0a316a2b69af349b` and `d9204b84f5f17b9205b6ec6aaaca3e8ee2b1b1f7e6534bcb3fe6cdf5e2c04818`.

12.5 Dependent-proof ranking from update zero

The next experiment removes the continuation history and changes the task. A small $84 \to 64 \to 1$ tanh ranker scores candidate refinements generated by the machine-checked dependent-proof frontier. Its training corpus contains 3,005 rows and 13,545 states; selection uses 600 family-held-out synthetic problems plus four external tutorial problems. All profiles consume the same 1,060-batch ledger, and PC is active from update zero rather than being initialized by BP.

The two-layer credit boundary was proved before training. For prospective output settling, both local parameter credits equal their BP counterparts at the same rate–precision scale:

```
theorem prospectiveFirstCredit_eq_scaledBP
    (precision rate firstWeight secondWeight input target : ℝ) :
  prospectiveFirstCredit precision rate firstWeight secondWeight input target =
    (precision * rate) * bpFirstCredit firstWeight secondWeight input target

theorem prospectiveSecondCredit_eq_scaledBP
    (precision rate firstWeight secondWeight input target : ℝ) :
  prospectiveSecondCredit precision rate firstWeight secondWeight input target =
    (precision * rate) * bpSecondCredit firstWeight secondWeight input target
```

The boundary is nontrivial: hidden-state PC can recover the first-layer BP credit exactly while its two-layer credit pair is not any common scalar multiple of BP (`hiddenPC_twoLayerCredit_not_commonScaledBP`).

The one-world screen separates fit from mechanism:

profile	held-out solves / 604	final loss	raw cosine to BP
BP control	286	0.0227	1.0000
independent BP	220	0.0205	1.0000
error-coordinate PC	244	0.0338	0.9899
prospective PC	119	0.5855	0.0979
incremental PC	1	1.7612	0.7638

Thus error-coordinate PC is empirically BP-like in this ranker, prospective PC is strongly distinct but lower-yield, and incremental PC is neither a useful fit nor a competitive search guide. Prospective and incremental PC were then retrained from the independent-BP initialization. Prospective PC reproduced the mechanism result almost exactly: final loss 0.6073, median raw-credit cosine 0.0930, and optimizer-displacement cosine 0.3358. Incremental PC again barely moved ($1.7590 \to 1.7497$; cosines 0.7236/0.8549).

profile	synthetic / 600	tutorial / 4	external test / 3	DTTBench / 31
matched independent BP	220	0	3	1
prospective PC replica	160	2	3	2
incremental PC replica	0	1	3	1

Prospective PC loses 60 total synthetic solves to BP but adds 40 outside BP's synthetic solved set. Its DTTBench proofs are `False_elim` and `Iff_refl`; BP proves only `False_elim`. Synthetic solved-set Jaccard similarity across prospective seeds is 0.6163 (106 shared of 172 in union). The independent verifier re-executed all 1,276 replica searches, kernel-rechecked 172 solved traces and recompiled all three DTT candidates in their original declarations. Its SHA-256 is

`b191d1e779298233d932caf281333fde57c4d4feee79d09c5ac8217ceaf8844d.`

The external test and DTTBench endpoints had already been opened globally before the replica was trained. The models and protocol were frozen before this evaluation and received no endpoint-dependent changes, but these endpoint rows are fixed-benchmark replication rather than fresh unopened evidence. The reproducible credit departure and synthetic held-out marginals are the cleaner evidence for retaining prospective PC as a diversity arm. Total coverage and equal-compute opportunity cost still determine whether it earns production budget.

13 Certified finite settling and authenticated checkpoint geometry

The cost result above names the problem but does not force a fixed settling depth. The newer development separates three objects that an implementation can actually measure: a finite credit direction g_T, its certified error radius ε_T around the exact task credit $g_\star$, and the work required to obtain it.

Theorem 13.1 (Observable finite-credit gate)**.** *If $\|g_T - g_\star\| \le \varepsilon_T < \|g_T\|$, then $\langle g_\star, g_T\rangle > 0$. With a directional quadratic upper model, the stronger inequality*

$$\frac{\alpha C}{2} < \|g_T\|(\|g_T\| - \varepsilon_T)$$

certifies strict task descent for the step $-\alpha g_T$.

The corresponding proof terms are

- alignment: `finiteCredit_positiveAlignment`;
- task-loss composition: `finiteCredit_strictTaskDescent`.

For a contraction solver, the observable residual supplies

$$\varepsilon_T \;\le\; K\,\frac{\|z_T - S(z_T)\|}{1-q} + \|R(z_\star) - g_\star\|,$$

so early stopping remains explicitly approximate and retains the separate equilibrium mismatch. This is not the claim that a stopped iterate *is* the fixed point.

Work normalization is equally explicit. One update costs $c_0 + c_{\mathrm{sweep}}T$; a budget determines the number of complete rounds, and `cumulativeDecreaseLower_gt_iff` gives the exact crossover at which more cheap, lower-margin rounds have the stronger cumulative guarantee. Both boundaries are executable: negligible fixed overhead can favour eight one-sweep rounds over two four-sweep rounds (`lowOverhead_truncation_has_stronger_guarantee`), while large fixed overhead reverses the comparison. These are guarantees about a declared loss model, not claims about verified OEIS yield; the latter still requires matched GPU-time experiments.

Affine acceleration is not a stability certificate. The source-to-theorem atlas also isolates the normalization shared by extrapolation and Anderson/Pulay fixed-point acceleration. For arbitrary real normed state spaces, coefficients summing to one preserve every constant state and genuine fixed point, and the accelerated error is exactly the same weighted combination of the constituent errors. Its norm is bounded by the absolute coefficient mass. This is sharp: the affine weights $(2, -1)$ preserve constants yet turn two unit-bounded scalar errors into error 3. Nonnegative affine weights preserve a common radius, while signed acceleration requires an additional residual, restart, or acceptance safeguard. Affine preservation therefore licenses a solver transformation; it does not by itself license an accelerated PC training profile.

A terminal acceleration polynomial does not fix drift sensitivity. For a fixed quadratic, a finite accelerated schedule is summarized by its terminal residual polynomial. Under time-varying Hessians, however, the ordered realization matters: an insertion from a source eigenspace to a target eigenspace is transported by source-spectrum factors before its time index and target-spectrum factors afterward [20]. This product-rule algebra is now formalized for arbitrary finite memory. The coupled spectral-pair execution is exactly affine in the perturbation scale, and every nonzero finite-difference quotient is the recursively accumulated time-ordered response (`inv_smul_runSpectralPair_target_sub_zeroScale`). The realization dependence is strict: two scalar schedules have the same fixed target and source terminal products but insertion responses 5 and 7 (`same_fixed_terminal_different_insertion_response`). Therefore selecting an accelerated PC profile from its fixed-curvature terminal rate alone is unsound; the run must also bind a time-ordered curvature-drift budget. The formal result does not establish the source's Chebyshev–Jacobi constants or global nonlinear behaviour.

Changing sparse frontiers need a switching budget. A sparse error-coordinate solver also changes its active set while settling. With separate entry and exit thresholds and hysteresis gap $\delta = \theta_{\mathrm{in}} - \theta_{\mathrm{out}} > 0$, every switch after the first on a coordinate consumes at least δ of signal variation. Consequently, for one coordinate,

$$\delta\,(N_{\mathrm{switch}} - 1) \;\le\; \sum_t |s_{t+1} - s_t|,$$

as proved by `gap_mul_switches_after_first_le_variation`. The aggregate law requires one further observable. If N is total membership churn, D is the number of distinct coordinates that switched at least once, and V is the sum of their switch-event variations, then exact accounting gives

$$N = (N - D) + D, \qquad \delta\,(N - D) \le V.$$

The first identity and budget are kernel-checked in `gap_mul_reportedChurn_sub_distinct_le_totalSwitchVariation`. Subtracting D is necessary: a two-coordinate fixture in which each coordinate switches only once has positive churn and zero emitted-event variation, refuting any charge of δN without the distinct-coordinate count (`totalChurn_without_distinctCount_cannot_be_charged`). The gap itself is also essential: a valid zero-gap trace alternates too cheaply for a unit switch cost (`zeroGapChatter_refutes_unit_switch_cost`).

The current preregistered four-sweep diagnostic is deliberately negative. Pointwise top-5% membership had zero churn, whereas fixed 5%/10% hysteresis made 2,761 membership changes, reduced the active count from 6,807 to 4,046, and captured less final squared mass. That artifact reports neither D nor per-coordinate switch-event variation and was bound to the earlier scalar theorem source. It therefore does not instantiate the aggregate certificate and does not authorize a trainer change. Future runtime traces must emit all three quantities (N, D, V); only then can the variation theorem compose with the full-gradient fallback required by the sparse solver. Thus per-sweep top-k membership is not certified merely because each frozen mask is well behaved.

Proof-carrying frozen-cone specialization. When a PC adapter is attached to a frozen backbone, static computation can be residualized rather than repeated during every settling sweep. For a semiring-polynomial expression with explicitly separated static and dynamic variables, residualization preserves its value (`eval_residualize`) and every formal derivative with respect to a dynamic input (`eval_formalDeriv_residualize`). The construction performs real constant folding, rather than validating itself by definition: a mixed fixture specializes nontrivially, while `misclassification_breaks_value_and_credit` shows that caching a dynamic dependency can agree at one observation yet break both the next value and the credit. These theorems establish exact symbolic preservation. They do not establish a GPU speedup, a floating-point schedule, or whole-model applicability; those require the generated residual program to pass the executable replay gate.

From source tensors to exact-real geometry. The finite-credit theorem is useful only if its constants can be tied to the program that ran. The checkpoint-replay tranche decodes finite IEEE-754 binary32 words inside Lean, reconstructs the row-major affine maps of one authenticated $256 \to 64 \to 64 \to 64 \to 256$ residual adapter invocation, and composes the three hidden affine–SiLU replay errors into an exact-real mismatch radius. The bridge theorem `threeStageOutputMismatch_le_from_geometry` uses the decoded checkpoint matrices rather than Python-computed ideal values. A shared block-diagonal layer inherits the single-node operator bound without a node-count factor (`decodedSharedLinear_norm_le_singleNodeWeightL1`), and a cross-node-coupling fixture proves why this does not extend to arbitrary coupled layers.

The authenticated invocation theorem is `invocation0_full_geometry_mismatch_le`. Its scope is pointwise: it binds exact words, one invocation, and an exact-real regional calculation. It does not identify a BLAS accumulation schedule, prove a uniform floating-point model, bind the entire frozen parent, or establish efficacy. Those are separate endpoints of the certificate chain.

Naming criterion. Acceleration that preserves a fixed point is a solver result, not automatically a new learning rule. Conversely, a rule that changes the mechanism must separate from the registered

BP, PC, PC–ALM, and prior-art families under matched compute before it earns a new name. No current candidate meets that bar. Equivalence and collapse are evidence, not failures to be hidden.

14 Verdict

Not competitive in the completed drop-in experiments reported here: predictive coding as a from-scratch replacement for backpropagation on synchronous digital hardware. At small scale it is dominated at equal epochs and vastly dominated at equal wall-clock — consistent with the measured 1.6–15× per-step cost of settled inference over backprop [16]; at medium multi-generation scale it holds a stable two-thirds frontier deficit with erratic self-training dynamics; on the recurrent architecture, seven registered arms produced exactly one passing design, and it is staged from a BP-equivalent anchor rather than trained from random initialization. This is a scoped experimental verdict about the end-to-end tree and seq2seq regimes, not a theorem about every carrier, schedule, or hardware model. It does not extend to the frozen-parent adapter factorial of Section 12.3, where prospective PC starts with the adapter at update zero and leads the aggregate verified-coverage portfolio of the two-world experiment —though not of the mis-specified single-world branch recorded in Section 12.4, where the independent-BP opportunity control has the largest single-arm coverage (400); that branch is footnoted as botched and carries no evidential weight on the from-scratch question.

Surviving, with formal or measured support:

1. *Shallow PC adapters under an exact frozen restriction.* The depth pathology depends only on the adapter after the coupled operator is genuinely restricted (Theorem 7.3), and the empirical verdict independently points at this regime. In the sealed two-world typed-GSLT factorial, prospective PC has the largest update-rule union (968) and exclusive marginal (299), though not a universal per-cell win and at substantially greater training cost; the subsequent mis-specified single-world branch (recorded, not evidential: Section 12.4) logged the reverse ordering. The theorem does not cover a shared active normalization or partial freezing: its negative half proves that an active unit-coupled backbone re-imports the backbone depth bound.

2. *BP-equivalent ignition followed by measured departure.* The staged homotopy arms were the only seq2seq arms to reach 32/32 exact decodes; the deepened-inference continuation passed every bar on seed 0, and the pre-registered independent-seed replication passed on seed 1 with distinctness at every checkpoint of a fixed window. This establishes one reproducible ignition route; it is no longer the only positive PC regime in the paper. It has the same amortize-then-refine structure that hybrid predictive coding uses to infer fast and then settle slow [12].

3. *PC-from-update-zero as a verified diversity arm.* In the dependent-proof screen of Section 12.5, prospective PC reproduced a raw-credit cosine near 0.09 on two seeds, contributed 40 synthetic held-out solutions outside its matched BP control and one additional fixed-benchmark DTT theorem, while losing on total synthetic coverage. This licenses continued evaluation alongside an independent-BP opportunity control; it does not license replacing BP. Incremental PC failed this promotion test unchanged.

4. *Second-order geometry, with the bar set by Theorem 8.5.* The preconditioning failures are exactly characterized, so the cure is exactly specified: a metric whose departure from the

identity is a contraction, $\rho(I - P) < 1$, on the modes that matter — a membership condition, not identity with backprop. Merely large, or merely uniform, is proven not to work.

5. *Rank-uniform skip architectures.* Theorem 5.3 says exact PC scheduling survives skip connections that do not mix target ranks — a strictly larger design space than levelizability, and untested.

6. *Asynchronous and neuromorphic settings.* Untested here; nothing in this note bears on them, except that the decisiveness instrument of Section 11 travels to any setting where a learned prior guides search. These are the settings the energy-based learning tradition, such as equilibrium propagation [18], was designed for, and where a settle-then-update rule is not paying a wall-clock penalty against a synchronous backprop baseline.

15 Limitations and open problems

The models are exact and, where it matters, small: scalar paths, finite priors, expected rather than at-least-one hit counts. Theorem 5.3 assumes strictly positive parent contributions; Theorem 5.2 now covers the signed case, so what remains open there is not the classification — Theorem 5.4 supplies that — but an architecture-level criterion for useful cancellations under realistic learned gains. Theorem 10.3 still needs reachability in its general form; Theorem 10.4 discharges it for one trained centered-sigmoid model, not for arbitrary coupled multivariate training. Theorem 8.3 gives finite-time nonnormal bounds but its constant is existential; sharp computable constants require more operator structure. Theorem 9.1 refutes universal strictness and classifies one critical line, not every equilibrated-energy critical manifold. The ePC equivalence is mathematical over exact reals; implementation-level rounding behavior remains a separate question. The ignition ladder is overfit-32 scale: its purpose is correctness-gated variant triage, not benchmark performance. The distinctness criterion's placement at the registered final step is a design choice whose alternatives remain untested. Settling itself is not unconditionally convergent: for large enough weights the relaxation can diverge [15], the empirical counterpart of the contraction condition our results assume. All "dead" verdicts are scoped to the regimes actually measured, and that scope has a provable outer boundary: in the infinite-width limit the distinctive predictive-coding dynamics collapse onto backpropagation [13], so the effect we are measuring is real precisely in the finite regime and vanishes where the network is wide relative to its depth. Depth in the other direction is likewise not a fundamental barrier: the degradation of standard predictive coding past a few layers has been traced to exponentially imbalanced inter-layer errors and repaired—by precision-weighted latent balancing, a deep-layer update correction, and auxiliary residual neurons—to backpropagation-comparable accuracy on deep residual networks [19], the algorithmic counterpart of the coordinate-dependence our error-coordinate section makes exact. We have since formalized those mechanisms at arbitrary depth: the first-arrival attenuation that produces the imbalance, the precision spike that exactly cancels it, the forward-reference update that bounds deep-layer accumulation, the auxiliary buffers that synchronize residual and main paths, and the invariance of frozen normalization statistics—over full covariance matrices with a nonlinear Jacobian remainder, not the scalar chain the rest of this note works in. The scope of that formalization should be read narrowly: it establishes exact local identities and propagation bounds, and it deliberately classifies convergence and accuracy for trained deep networks as empirical targets rather than theorems. What none of it repairs is the cost: the repaired dynamics remain an iterative relaxation, so "matches backpropagation" continues to mean matches its target, not its price.

A The causal boundary: clamping is not intervention

A.1 Observation is direction-blind

Take centered variables with positive-definite covariance

$$\Sigma = \begin{pmatrix} v_X & c \\ c & v_Y \end{pmatrix}, \qquad \Delta = v_X v_Y - c^2 > 0.$$

The fitted $X \to Y$ quadratic PC model has coefficient c/v_X and residual variance Δ/v_X; the reverse model has coefficient c/v_Y and residual variance Δ/v_Y. Their residual operators are provably distinct. Nevertheless, direct algebra reduces both full energies, pointwise for every real pair (x, y), to

$$\frac{v_Y x^2 - 2cxy + v_X y^2}{\Delta}.$$

The equality therefore lifts to every finite batch of *arbitrary* observations. There is no Gaussian data hypothesis: "Gaussian" names the quadratic energy. More generally, the formal batch theorem is

$$\sum_s E(z_s) = \operatorname{tr}(Q\widehat{\Sigma}), \qquad Q = A^\top \Lambda A, \quad \widehat{\Sigma} = \sum_s z_s z_s^\top.$$

Changing the data distribution while retaining this energy cannot expose higher-order orientation information. A non-quadratic residual log-density would change the model's architecture, not merely its environment.

Ordinary PC clamping does not help. Clamp $X = x$ but retain every residual term and minimize over Y. In both orientations the unique equilibrium is $Y = (c/v_X)x$, the ordinary conditional mean. PC clamping is conditioning, and conditioning is direction-blind.

A.2 One uniform severing operation

Pearl's intervention replaces the intervened structural mechanism rather than conditioning on its output [8, 9]. At the PC energy level this is one orientation-independent operation: clamp a node and delete the residual term owned by that node. Residual rows are indexed by their owning node, so the operation itself never inspects edge direction.

Theorem A.1 (Intervention certificate)**.** *Under $do(X = x)$, the $X \to Y$ equilibrium remains $(c/v_X)x$, whereas the $Y \to X$ equilibrium is 0. They differ if and only if $c \neq 0$ and $x \neq 0$. The two interventional energy functions are*

$$E^{do}_{X\to Y}(x, y) = \frac{(y - (c/v_X)x)^2}{2(\Delta/v_X)}, \qquad E^{do}_{Y\to X}(x, y) = \frac{y^2}{2v_Y},$$

and some intervention separates them if and only if $c \neq 0$.

Lean 4 signature and complete proof

```
theorem observationally_blind_intervention_certificate
    {Sample : Type*} [Fintype Sample]
    (covariance : PositiveBivariateCovariance)
    (hcorrelated : covariance.covXY ≠ 0) (x y : Sample → ℝ) :
    covariance.forwardResidualMatrix ≠ covariance.reverseResidualMatrix ∧
      (forwardNodeResidualModel covariance).residualMatrix ≠
        (reverseNodeResidualModel covariance).residualMatrix ∧
      (∑ sample, covariance.forwardGaussianPCEnergy (x sample) (y sample)) =
        ∑ sample, covariance.reverseGaussianPCEnergy (x sample) (y sample) ∧
      covariance.SeparatesDirectionsByInterventionalEnergy
```

At the concrete covariance $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ and $x = y = 1$, both half-normalized observational energies equal $1/2$, while the interventional energies are respectively 0 and $1/4$. The negative fixture is just as important: independent equal-variance variables give $y^2/2$ in both orientations, so intervention does not orient an absent edge. This certificate supplies one bit about one edge; it is not a general causal-discovery theorem.

An energy difference at a selected state is not yet a learning theorem, so we also let both orientations fit. On a finite intervention design $(x_s)_s$, take responses from the true forward mechanism, $y_s = (c/v_X)x_s$. A forward candidate fits its own coefficient β and a reverse candidate fits its own coefficient γ; both are scored by the same severing operation. Their batch objectives reduce to

$$L_{X\to Y}(\beta) = \frac{(c/v_X - \beta)^2}{2(\Delta/v_X)} \sum_s x_s^2, \qquad L_{Y\to X}(\gamma) = \frac{(c/v_X)^2}{2v_Y} \sum_s x_s^2.$$

The reverse candidates are structurally distinct as γ varies, but their fitted row is precisely the row deleted by $do(X)$; the second objective's independence from γ is therefore a theorem about the uniform intervention, not a restriction of the competitor.

Theorem A.2 (Fitted two-node causal selection)**.** *For any finite design with $\sum_s x_s^2 > 0$, both orientation-specific objectives attain their minima. The minimized forward energy is strictly lower than the minimized reverse energy if and only if $c \neq 0$.*

Lean 4 signature and complete proof

```
theorem forwardOrientationWinsByMinimizedInterventionalEnergy_iff
    {Sample : Type*} [Fintype Sample]
    (covariance : PositiveBivariateCovariance) (x : Sample → ℝ)
    (hexcited : 0 < ∑ sample, x sample ^ 2) :
    covariance.ForwardOrientationWinsByMinimizedInterventionalEnergy x ↔
      covariance.covXY ≠ 0
```

Indeed, $\beta = c/v_X$ gives forward minimum zero, while every γ minimizes the reverse objective at the displayed positive value exactly when $c \neq 0$. At independent equal variance both minima are zero, so selection fails. The conclusion is strictly two-node and assumes interventional responses from the forward linear mechanism, positive residual scales, and an excited finite design; it is not a graph-discovery claim.

A.3 A separate continual-learning bridge

Another causal-coding proposal turns on whether learning one thing and then another is the same as learning both. It is not, and the defect is exactly computable in the quadratic case. For two quadratic tasks with curvatures A_1, A_2, step size η, and gradient g_1:

$$\theta_{1\to 2} - \theta_{\text{add}} \;=\; \eta^2 A_2\, g_1(\theta), \qquad \theta_{1\to 2} - \theta_{2\to 1} \;=\; \eta^2 [A_1, A_2]\, \theta,$$

the second for centred tasks. So sequential learning departs from additive learning by the second task's curvature acting on the first task's gradient, and the order in which you learn matters exactly to the extent that the curvatures fail to commute.

These are two different phenomena, and we separate them rather than let one stand in for the other. A fixture with oblique tasks has a non-zero commutator and is order-dependent. A fixture in which one task is repeated has a *zero* commutator — no order interference is possible — and is still non-additive, because the same cause is counted twice. On the evidence side that double counting is exact: re-applying a Gaussian contribution yields the correct once-each ledger plus precisely one extra copy of that contribution.

We state the scope plainly, and the formalization enforces it: what is proven is linear and quadratic. The general correspondence between predictive coding and causal coding remains explicitly conjectural beyond these instances, and its status now records that a causal use requires an interventional clamp. No theorem promotes the general correspondence.

A.4 Forgetting has a metric side and a connection side

The two phenomena above are instances of a general decomposition that organizes a recent coupling/curvature synthesis of continual learning: forgetting is a sum of a *metric* part (some directions receive the wrong effective weight) and a *connection* part (task updates transported along a curriculum do not commute), plus a remainder. The metric side is already the subject of this paper—the localization inequality of the cost-of-depth section is a reach-versus-conditioning statement, and over-settling is a scheduling defect. We formalized the connection side to the same standard, and the exercise both confirmed the sound core of the synthesis and corrected two of its claims.

Confirmed. For two rank-one task curvatures the interference is a clean trichotomy (`rankOne_parallel_orthogonal_oblique_trichotomy`): parallel tasks commute and any forgetting is pure double counting, a metric effect; orthogonal tasks do not interact; oblique tasks have a non-zero commutator, which is the irreducible curvature. The per-pair conflict is measured exactly by the diagonal of an interference Gram matrix built from commutators, and that diagonal vanishes if and only if the pair commutes (`pairwiseInterferenceEnergy_eq_zero_iff_commute`); adding a scalar multiple of the identity to a curvature leaves it unchanged, so absolute damping is invisible to it, as the synthesis predicts. On the transfer side, an invertible similarity conjugates the commutator and preserves its characteristic polynomial (`matrixCommutator_charpoly_unitConjugate`): interference *spectra* are portable across a first-derivative-level equivalence of two systems, while pointwise agreement of values and gradients does not determine curvature at all (`pointwiseAgreement_not_determineHessian_negativeExample`)—behavioural similarity is not enough to trust a transferred forgetting measurement.

Corrected. Two natural-looking claims are false as stated, and the counterexamples are machine-checked. First, "the curriculum's rotational factor is trivial exactly when the tasks commute" holds for two symmetric exponential factors (`twoSymmetricFactors_trivialRotationProxy_iff_commute`) but *not* in general: a palindromic product $A\,B\,A$ of symmetric matrices is symmetric even when A and B do not commute, so a trivial rotational proxy does not imply commutation for arbitrary curricula. Second, interference *angles*—as opposed to the spectrum—are preserved only under *orthogonal* change of coordinates, not arbitrary similarity: an explicit shear conjugates a symmetric operator to one with different rotational data (`nonorthogonalSimilarity_not_preserve_trivialRotationProxy_negativeExample`). The distinction matters in practice: a curriculum's interference eigenvalues transfer under a derivative-level equivalence, but its interference geometry transfers only under the stronger orthogonal one.

The scope is again exactly linear and quadratic, and the localization inequality in particular remains a physical analogy with unpinned constants until analytic hypotheses are supplied; the

formalization records that boundary rather than asserting past it (`sealedLocality_requires_distance_le_sweeps_mul_bandwidth`). What the connection side offers a synthesis loop is a diagnostic: the interference Gram of successive generations measures whether they commute, and hence whether the order in which a self-improving learner accumulates its own discoveries can matter—an empirical question these invariants make measurable. The same invariant does double duty on the architecture side: order-independence of a routed decoder's command phases holds exactly when the pairwise interference energy of their generators vanishes (`linearPhases_orderIndependent_iff_interferenceEnergy_zero`), so one Gram matrix certifies both a curriculum's right to be reordered and a command schedule's right to be treated as a set.

A.5 Predictive coding as a frozen-backbone residual cap

A last practical question is whether predictive coding earns a place not as a whole-network learning rule—where the depth results above make it a rescaling of backpropagation with a rescaling's pathologies—but as a shallow, locally trained residual adapter between a frozen backbone and a symbolic action head, the one regime the depth theorems endorse. We tested this prospectively: a matched two-link residual adapter, trained by backpropagation versus by a local predictive-coding update, over *both* a recurrent and a typed-workspace frozen backbone, three paired seeds, one cumulative generation of exact whole-corpus verification. The adapter's local update reproduced the backpropagation gradient at its settling limit to cosine 1.0000 before the run, so the arms differ only by the rule. The registered result is neutral: pooled new verified sequences were 57 versus 54 on the recurrent backbone and 59 versus 60 on the workspace backbone, an interaction of +4 carried entirely by one of three seeds. Shallow predictive coding as a frozen-backbone cap is thus neither the liability the full-network depth results warn against nor an advantage; it is, at this scale, dominated-in-simplicity by the backpropagation adapter it matches.

A.6 The full credit-rule protocol

The 2×2 above is one slice of a registered longitudinal comparison whose full protocol we state here, since the preceding sections leave it implicit. The comparison separates two regimes. In the *full-model* regime—the tree network of Section 12.1 and the sequence-to-sequence ladder of Section 12.2—predictive coding is the whole learner, and it is there that it is costwise dominated. In the *frozen-adapter* regime a backbone is frozen and every rule trains only a zero-initialised residual adapter $256-64-64-64-256$ (three hidden sites), inserted so the backbone's output is preserved exactly at initialisation. Five rules share that adapter, differing only in the update:

Rule	Mechanism	T / rate / precision	cos→BP
BP control	AdamW backpropagation on the adapter	0	1.000
BP+	task + proximal $\|\theta - \theta_{\text{ref}}\|^2$, retention projection	0	≈1
prospective PC	settle the output-residual latent, adapter chases it	8 / 5×10^{-4} / 1000	0.99999
incremental PC	one latent and one adapter step, interleaved	4 / 10^{-3} / 300	0.011
deep ePC	error units at three sites, detached between blocks	4 / 10^{-3} / 300	0.99973

Two facts govern any comparison of the arms. First, at their selected mechanism-gate configurations prospective PC and deep ePC are backpropagation-like by construction (cosine to the backpropagation gradient ≈1); only incremental PC is genuinely distinct at selection (cosine 0.011), and the prospective cell reaches distinctness only by drifting over full training. Second, a code oracle

proves the corner exactly: with one settling step and rate × precision = 1 the adapter's local gradient equals the backpropagation gradient. The classification bins are backpropagation-like ≥ 0.995, intermediate 0.95–0.995, distinct < 0.95.

The design is a $2 \times 4 \times 5$ factorial—two hash-seeded worlds; four state carriers (recurrent, typed workspace, belief-derived, belief-learned); five rules—run longitudinally over generations, each cell effect read against its within-carrier backpropagation control and each architecture-by-rule interaction against the recurrent arm. Every cell shares a byte-identical frozen parent and adapter initialisation; a cell passes only if the parent state hash is unchanged after training and zero-adapter output parity holds under exact tensor equality. The optimiser is AdamW at 3×10^{-4} over 2500 updates of batch 16 (incremental PC dividing its rate by T), all PC inference using monotone backtracking line search (shrink 0.5, at most twelve backtracks, tolerance 10^{-6}). The completed, independently adjudicated generation-nine/ten coverage is reported in Section 12.3.

Where to find these

All statements are machine-checked in Lean 4, in the Mettapedia library under the namespace `Mettapedia.MachineLearning`. Each is axiom-clean, depending only on `propext`, `Classical.choice` and `Quot.sound`. Files below sit in `NeuralNetworks/PredictiveCoding/` unless marked otherwise.

Table 1: Index of formal statements.

	Lean name	File
Thm 3.1	`LinearGaussianOperatorModel.equilibrium_iff_eq_posteriorMean`	LinearGaussian-Operator
	`matrixPCEquilibrium_iff_eq_conditionalPosteriorMean`	MatrixGaussianChain
	`matrixPC_not_equilibrium_of_ne_posteriorMean`	(negative fixture)
	`pcConditionalPosteriorMean_equilibrium`	LinearGaussianChain (scalar)
Thm 4.1	`equilibriumError_satisfies_backpropRecursion`	BackpropExactness
	`tiedEquilibriumError_satisfies_bpttRecursion`	(weight tying)
Thm 4.2	`sharedDAGEquilibriumError_satisfies_parentAggregationRecursion`	SharedLatentDAG
	`dagMultiParent_parentAggregate_counts_both_slots`	(positive fixture)
	`dagMultiParent_single_slot_is_incorrect_negative_example`	(negative fixture)
Thm 4.3	`equilibriumGradient_ne_backpropGradient_example`	BackpropExactness
	`equilibrium_state_ne_forward_state_example`	(root cause)
Thm 5.1	`zilScheduledGainGradient_eq_backpropGradient`	ZILExactness
	`zilChainResidual_at_schedule_eq_backpropError`	(front lemma)
	`zilOffSchedule_freeEquilibriumGradient_failure`	(negative)
	`residualSkip_naiveFirstArrivalGradient_failure_example`	(negative)
Thm 5.2	`dagScheduleExactAtEveryNode_iff_all_missed_eq_zero`	DAGSchedule-Exactness
	`dagScheduleExactAtEveryNode_iff_each_admissible_or_cancellation`	
	`dagSchedule_exact_not_admissible_iff_hasMissedCancellation`	
	`dagScheduleExactAtEveryNode_iff_admissible_of_noCancellation`	(fusion under no cancellation)
Thm 5.4	`dagScheduleUniversallyExactAtEveryNode_iff_targetGain_zero`	DAGSchedule-Exactness
	`dagFirstArrivalScheduleUniversallyExact_iff_targetGain_zero`	
	`moduleScheduleUniversallyExactAtNode_iff_targetGain_zero`	ModuleSchedule-Exactness
	`moduleSignedCancellation_universallyExact_positive_example`	(positive fixture)

	Lean name	File
	`moduleSingleMissedEdge_not_universallyExact_negative_example`	(negative fixture)
Thm 5.3	`dagFirstArrivalScheduleExact_iff_uniformOutgoingTargetRanks`	DAGSchedule-Exactness
	`uniformLongSkipGraph_firstArrival_exact`	(witness)
	`uniformLongSkipGraph_not_unitLevelized`	(witness)
Thm 6.1	`spcErrorStream_epcStateStream`	ErrorState-Reparameterization
	`epcStateStream_spcErrorStream`	
	`epcPrefixEnergy_eq_spcPrefixEnergy`	
	`epcCriticalDifferential_iff_spcCriticalDifferential`	
	`epcLocalParameterUpdate_eq_spcLocalParameterUpdate`	
Thm 6.2	`spcFirstArrivalSignal_layer_closedForm`	ErrorState-Reparameterization
	`spcWavefront_eventually_below_tolerance`	
	`spcWavefront_nonzero_but_below_tolerance_example`	(positive)
	`spcWavefront_unitRate_not_below_small_tolerance`	(boundary)
	`epcOneStepParameterUpdate_unitRate_eq_backprop`	
Thm 7.4	`spcFirstArrivalParameterUpdate_eq_depthCreditBackprop`	MuPCDepthCredit
	`chainLinkLoss_after_uniformDepthCredit_exact`	
	`inferenceRateEqualization_forces_unitBackpropBoundary`	
	`muPCDepthScaling_cannot_cancel_contractingDepthCredit`	
	`uniformPrecision_equilibrium_sub_forward_exact`	
Thm 7.1	`pathSlowMode_eigenrelation`	DepthPathology
	`pathSlowMode_iterate_exact`	
	`pathRelaxationRate_gap_bound`	
	`pathRelaxationRate_tendsto_one`	
Thm 7.2	`HeterogeneousDepthBlockFamily.modalSpectralRadius_lt_one_iff`	Heterogeneous-DepthBlocks
	`HeterogeneousDepthBlockFamily.modalSpectralGap_le_depth_bound`	
Thm 7.3	`coupledBackboneAdapterStep_not_blockDiagonal`	FrozenAdapter-Restriction
	`frozenAdapterRestriction_eq_blockPathJacobiStep_clamped`	
	`frozenAdapterRestriction_eq_blockPathJacobiStep_of_boundary_clamped`	
	`isFrozenAdapterSlowModeRate_iff`	
	`frozenAdapterRestriction_gap_bound`	
	`activeBackboneAdapter_backbone_gap_bound`	
	`partialFreezing_reimports_backbone_negative`	
Thm 8.1	`chainLinkLoss_preconditionedUpdate_exact`	Preconditioned-Learning
	`chainLinkResidual_preconditionedUpdate_exact`	
	`chainLink_zeroLoss_afterUpdate_iff_identity`	
	`chainLink_preconditionedUpdate_strictDescent_iff`	
	`chainLink_lossDecrease_gap_from_identity_exact`	
	`chainLink_lossDescent_strictly_degrades_with_squaredDeparture`	
	`unitChain_large_preconditioner_increases_loss_negative_example`	
Thm 8.2	`spectralRadius_lt_one_iff_pow_tendsto_zero`	OperatorStability
	`complexMatrixPreconditioner_fullSpectrum_convergence`	
	`nonnormalTransientMatrix_fullSpectrum_stable`	(nonnormal witness)
	`nonnormalTransientMatrix_not_normal`	
Thm 8.3	`operatorPowNorm_geometricEnvelope`	OperatorStability
	`operatorResidualIterate_norm_geometricEnvelope`	
Thm 8.4	`jordanTransientMatrix_pow_exact`	OperatorStability

	Lean name	File
	jordanTransientMatrix_spectrum	
	jordanTransientMatrix_not_normal	
	spectralRadius_zero_allows_arbitrary_transientAmplification	(negative boundary)
Thm 8.5	matrixPreconditionedResidual_loss_perMode_exact	PreconditionedLearning
	mixedModePreconditioner_helps_one_hurts_another	
Thm 8.6	armijoBacktracking_terminates	BacktrackingDescent
	armijoCondition_energy_nonincreasing	
	pcEnergyDepth2_armijoBacktracking_terminates_with_descent	
Thm 9.1	orthogonalDataset_outputDirection_curvature	EquilibratedEnergySaddles
	orthogonalDataset_innerStrip_hasStrictSaddleCertificate	
	orthogonalDataset_counterexamplePoint_all_curvatures_zero	
	orthogonalDataset_counterexamplePoint_twoSidedSaddleCurves	
	orthogonalDataset_universalStrictSaddleConjecture_counterexample	(negative boundary)
Thm 10.1	coupledTwoTimescale_multipliers_stable_iff	CoupledDynamics
	coupledTwoTimescale_converges_below_threshold	
	coupledTwoTimescale_diverges_above_learning_threshold	
Thm 10.2	arbitraryDepthLinearEntropyTurnover_general	EntropyTurnover
	arbitraryDepthTurnoverEntropy_nonmonotone_iff	
Thm 10.3	monotoneContractiveNonlinearEntropyTurnover_sufficient	EntropyTurnover
	MonotoneContractiveSettling.iterate_abs_le	(contraction bound)
	monotoneContractiveNonlinearEntropyTurnover_uniform_boundary	(boundary)
Thm 10.4	trainedCenteredSigmoidWeight_is_unique_global_minimizer	TrainedNonlinearTurnover
	trainedCenteredSigmoidEntropyTurnover	
	approximateTraining_preservesCenteredSigmoidTurnoverMargin	
	centeredSigmoidContraction_does_not_reach_logit_one	(boundary)
	trainedCenteredSigmoidVectorTurnover	(vector lift)
	centeredSigmoidVectorZeroSeed_uniform	(negative fixture)
Appendix	PositiveBivariateCovariance.distinct_orientations_same_observational_batchEnergy	CausalDirectionIdentifiability
	PositiveBivariateCovariance.clamp_is_conditioning_and_direction_blind	
	PositiveBivariateCovariance.interventionalEquilibria_separate_iff	
	PositiveBivariateCovariance.observationally_blind_intervention_certificate	
	PositiveBivariateCovariance.TwoNodeQuadraticResidualModel.batchEnergy_eq_trace_quadraticForm_mul_empiricalSecondMoment	
Thm A.1	PositiveBivariateCovariance.separatesDirectionsByInterventionalEnergy_iff	CausalDirectionIdentifiability
	PositiveBivariateCovariance.unitNoiseForwardCovariance_observation_tied_intervention_separates	(positive fixture)
	PositiveBivariateCovariance.independentEqualVarianceCovariance_interventionalEnergy	(negative fixture)
Thm A.2	PositiveBivariateCovariance.forwardOrientationWinsByMinimizedInterventionalEnergy_iff	CausalDirectionSelection
	PositiveBivariateCovariance.isMinimizedForwardInterventionalBatchEnergy_iff	(unique forward fit)
	PositiveBivariateCovariance.reverseCandidates_distinct_but_intervention_deletes_parameter	(anti-vacuity)

	Lean name	File
	`PositiveBivariateCovariance.unitNoiseForwardCovariance_minimizedIntervention_selects_forward`	(positive fixture)
	`PositiveBivariateCovariance.independentEqualVariance_minimizedIntervention_tied`	(negative fixture)
Appendix	`sequentialTwoTaskUpdate_sub_additive_exact`	ContinualLearning/ QuadraticTwoTask
	`centered_sequential_order_defect_eq_commutator`	
	`linearTwoTask_causalCoding_separation_crown`	
	`GaussianEvidence.reused_contribution_exact_excess`	ContinualLearning/ EvidenceLedger
	`linearCausalCodingBridge_crown`	
	`generalCausalCodingCorrespondence_requires_interventionalClamp`	(scope marker)
Thm 11.1	`iidExpectedTargetHits_eq_budget_mul_targetMass`	SearchGuidance/ DecisivenessYield
	`iidExpectedTargetHits_strictMono_targetMass`	
Thm 11.2	`equal_crossEntropy_strictly_ordered_expectedYield`	SearchGuidance/ DecisivenessYield
Sec. 12.5	`DTTTwoLayerPC.prospectiveFirstCredit_eq_scaledBP`	CreditTransport/ DTTTwoLayerPC
	`DTTTwoLayerPC.prospectiveSecondCredit_eq_scaledBP`	
	`DTTTwoLayerPC.hiddenPC_twoLayerCredit_not_commonScaledBP`	(negative boundary)
	`AdaptivePortfolio.insert_candidate_gt_insert_opportunity_iff`	SearchGuidance/ CostAwareOpportunityControl

Related work

Equivalence-in-the-limit results and their scope: [1] (MLPs), [3] (arbitrary graphs, including LSTMs at small scale), [2] (exact scheduling), [4] (topological limits). Engineering of deep PC: [5], whose diagnosis that naive settling misbehaves in digital simulation Theorems 7.1 and 8.1 corroborate from the analytic side. Equilibrated-energy landscape results and the universal strict-saddle conjecture are due to [6]; Section 9 proves a degenerate boundary outside their covered class. The program-synthesis substrate is Gauthier's OEIS self-learning system and its NMT variant [10]. Our companion report *AlienCoding* documents the replication methodology, the full campaign, and the reference numbers.

References

[1] J. Whittington and R. Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*, 29(5), 2017.

[2] Y. Song, T. Lukasiewicz, Z. Xu, and R. Bogacz. Can the brain do backpropagation? Exact implementation of backpropagation in predictive coding networks. *NeurIPS*, 2020.

[3] B. Millidge, A. Tschantz, and C. Buckley. Predictive coding approximates backprop along arbitrary computation graphs. arXiv:2006.04182, 2020.

[4] T. Salvatori et al. Learning on arbitrary graph topologies via predictive coding. *NeurIPS*, 2022.

[5] C. Goemaere, G. Oliviers, R. Bogacz, and T. Demeester. ePC: Fast and deep predictive coding in digital simulation. arXiv:2505.20137, 2025.

[6] F. Innocenti, E. M. Achour, R. Singh, and C. L. Buckley. Only strict saddles in the energy landscape of predictive coding networks? *Journal of Statistical Mechanics: Theory and Experiment*, 2025. doi:10.1088/1742-5468/ade2eb.

[7] F. Innocenti, E. M. Achour, and C. L. Buckley. μPC: Scaling predictive coding to 100+ layer networks. arXiv:2505.13124v2, 2025.

[8] J. Pearl. The do-calculus revisited. In *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*, 2012.

[9] J. Y. Halpern. Axiomatizing causal reasoning. *Journal of Artificial Intelligence Research*, 12:317–337, 2000.

[10] T. Gauthier, M. Olšák, and J. Urban. Alien coding. arXiv:2301.11479, 2023.

[11] B. Millidge, Y. Song, T. Salvatori, T. Lukasiewicz, and R. Bogacz. A theoretical framework for inference and learning in predictive coding networks. arXiv:2207.12316, 2022.

[12] A. Tschantz, B. Millidge, A. K. Seth, and C. L. Buckley. Hybrid predictive coding: Inferring, fast and slow. *PLoS Computational Biology*, 2023; arXiv:2204.02169.

[13] F. Innocenti et al. Infinite width and depth limits of predictive coding networks. arXiv:2602.07697, 2026.

[14] A. Ofner et al. Predictive coding, precision and natural gradients. arXiv:2111.06942, 2021.

[15] S. Frieder and T. Lukasiewicz. (Non-)convergence results for predictive coding networks. *International Conference on Machine Learning*, PMLR 162, 2022.

[16] R. Rosenbaum. On the relationship between predictive coding and backpropagation. *PLoS ONE*, 2022; arXiv:2106.13082.

[17] L. Pinchetti et al. Predictive coding beyond Gaussian distributions. *NeurIPS*, 2022; arXiv:2211.03481.

[18] B. Scellier and Y. Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.

[19] C. Qi, M. Forasassi, T. Lukasiewicz, and T. Salvatori. Towards the training of deeper predictive coding neural networks. arXiv:2506.23800, 2025.

[20] D. Pasechnyuk-Vilensky and M. Takáč. Chebyshev-exact acceleration under Hessian variation, I: Sine–Jacobi method. arXiv:2606.16671, 2026.